NLP Lecture 2026

Traitement du Langage Naturel

Christophe Ambroise

2026-03-14

Table of contents

1 NLP Lecture 2026 **1**
1.1 Contents . . . 1
1.2 Learning objectives . . . 1

2 Word Embeddings **3**
2.1 Different approaches of language modeling . . . 3
2.2 History of Word Embeddings . . . 4
2.3 From Words to Vectors . . . 4
2.4 Word-document matrix . . . 5
2.5 Word-Word matrix . . . 5
2.6 Cosine for measuring similarity . . . 6
2.7 TF-IDF: Weigthing terms in the vector . . . 6
2.8 TF-IDF: Weigthing terms in the vector . . . 7
2.9 Pointwise Mutual Information (PMI) . . . 7
2.10 Positive PMI (called PPMI) . . . 7
2.11 Example of PPMI weighting . . . 8
2.12 Applications of the TF-IDF or PPMI vector models . . . 8
2.13 Latent Semantic Analysis (LSA) . . . 9
2.14 Latent Semantic Analysis . . . 9
2.15 Latent Semantic Analysis . . . 10
2.16 Latent Dirichlet Allocation (LDA) . . . 10
2.17 LDA Generative process . . . 10
2.18 Latent Dirichlet Allocation (LDA) in summary . . . 11
2.19 Latent Dirichlet Allocation (LDA) estimation . . . 11
2.20 Example of LDA in Action . . . 11
2.21 Pro and Cons . . . 12
2.22 Word2vec . . . 12
2.23 Embedding derived from classification . . . 13
2.24 From dot product to logistic regression . . . 13
2.25 Skip-gram training pairs . . . 14
2.26 Two-matrix architecture . . . 14
2.27 Final embeddings . . . 14
2.28 GloVe: from local windows to global statistics . . . 15
2.29 GloVe objective . . . 15
2.30 Interpretation of the GloVe objective . . . 15
2.31 GloVe: meaning from co-occurrence ratios . . . 16
2.32 Word2Vec vs GloVe . . . 16
2.33 Analogy Tasks . . . 17

3 Exercises — LSA and Word2Vec Embeddings **19**
3.1 Links . . . 19

4 Embeddings **23**
4.1 Part 1 — Latent Semantic Analysis (LSA) . . . 23
4.2 Part 2 — Word2Vec . . . 33
4.3 Part 3 — Comparison: LSA vs Word2Vec . . . 35
4.4 Summary: LSA vs Word2Vec . . . 37

5 Transformers **39**
5.1 Outline . . . 39
5.2 From Turing to ChatGPT . . . 39
5.3 What is a Transformer? . . . 40
5.4 Optimized criteria . . . 40
5.5 Masked Language Modeling (MLM) . . . 41
5.6 Conditional Language Modeling / Translation Loss . . . 41
5.7 Initial embedding . . . 41
5.8 A minimal self-attention architecture for decoder . . . 42
5.9 Attention and matrix notation . . . 42
5.10 Illustration of self-Attention . . . 43
5.11 Interpretation of Q, K, V . . . 43
5.12 Important details . . . 44
5.13 Layer Normalization . . . 44
5.14 Dropout in Transformers . . . 44
5.15 Positional encoding . . . 45
5.16 Feed Forward Neural Network . . . 45
5.17 Skip connection (original paper "Deep Residual Learning for Image Recognition" 2015 by He et al.) . . . 45
5.18 Multi-head attention . . . 46
5.19 Complexity of Self-Attention . . . 46
5.20 Three Transformer Families . . . 47
5.21 Tokenization . . . 47
5.22 Fine tuning . . . 47
5.23 Generating Text from a Decoder . . . 48
5.24 From Theory to Code . . . 49

6 Exercises — GPT from scratch **51**
6.1 Links . . . 51
6.2 Supplementary links . . . 51

7 NanoGPT **57**
7.1 Comprehension Questions — Tokenization . . . 59
7.2 Comprehension Questions — Training Objective . . . 60
7.3 Comprehension Questions — Bigram Language Model . . . 62
7.4 The mathematical trick in self-attention . . . 64
7.5 Comprehension Questions — Weighted Aggregation via Matrix Multiplication . . . 65
7.6 Comprehension Questions — Self-Attention . . . 66
7.7 Comprehension Questions — Layer Normalization . . . 68
7.8 Comprehension Questions — Full GPT Model . . . 68

List of Figures

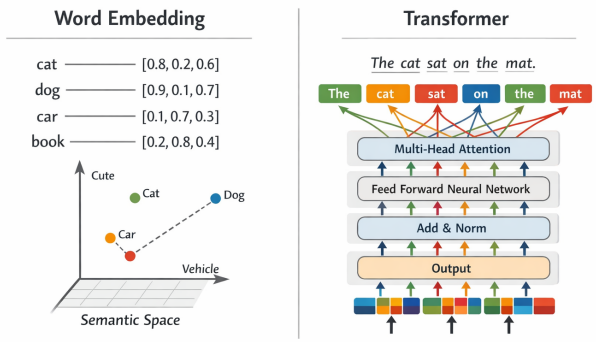
5.1 History of AI leading to chatGPT 39
5.2 The nature of transformer 40
5.3 Initial embedding 42
5.4 Self-attention block in auto-encoder 43
5.5 Skip connection 45
5.6 ChatGPT fine tuning (from A. Karpathy lecture) 48

List of Tables

2.1 Term-document matrix — S1: *"word embeddings represent meaning"* · S2: *"deep neural networks learn representations"* · S3: *"language models process text"* · S4: *"word vectors capture context"* · S5: *"neural language models predict"* 5
2.2 Word-word co-occurrence matrix (window $L = 1$) 5
2.3 TF-IDF weighted term-document matrix — TF $= \log_{10}(1 + 1) = 0.30$ for all present terms (each appears once per sentence) 7
2.4 PPMI matrix — all other co-occurrences have PMI ≤ 0, set to 0 by PPMI 8

Chapter 1

NLP Lecture 2026



Natural Language Processing

This document covers the fundamentals of NLP, from word embeddings to Transformer architectures, with practical exercises covering implementation and applications.

1.1 Contents

1. **Word Embeddings** - Word vectors, co-occurrence matrices, TF-IDF, PMI, LSA, LDA, and Word2Vec
2. **Transformers** - Attention architecture, contextual embeddings, and language models
3. **Exercises** - Practical implementation

1.2 Learning objectives

- Master different approaches to word embeddings
- Understand attention mechanisms and Transformers
- Implement the concepts with practical code

Course 2026

Chapter 2

Word Embeddings

2.1 Different approaches of language modeling

Three paradigms have shaped how machines process and understand language:

2.1.1 Symbolic (rule-based)

Hand-crafted grammars, lexicons, and inference rules encode linguistic knowledge explicitly.

- **How it works**: linguists write rules (*"a noun phrase = determiner + adjective + noun"*); parsers apply them mechanically.
- **Strengths**: interpretable, deterministic, works well on narrow well-defined tasks.
- **Weaknesses**: brittle — rules break on unseen text; enormous engineering effort; poor coverage of natural language variability.
- **Examples**: ELIZA (1966), WordNet, classic machine translation systems (SYSTRAN).

2.1.2 Statistical (data-driven)

Learn probability distributions over words and sequences from large corpora; no hand-crafted rules.

- **How it works**: count co-occurrences, estimate n-gram probabilities, or learn latent factors (TF-IDF, PMI, LSA, LDA).
- **Strengths**: robust to variation, scales with data, language-agnostic.
- **Weaknesses**: requires large corpora; bag-of-words models lose word order; sparse representations struggle with rare words.
- **Examples**: n-gram language models, TF-IDF retrieval, LSA, LDA.

2.1.3 Neural (deep learning)

Learn dense, distributed representations end-to-end from raw text using differentiable architectures.

- **How it works**: train neural networks (feedforward, RNN, Transformer) on prediction tasks; representations emerge as by-products of training.
- **Strengths**: capture long-range dependencies, share statistical strength across similar words (dense embeddings), generalise well.
- **Weaknesses**: data- and compute-hungry; less interpretable; static embeddings conflate word senses.

- **Examples**: Word2Vec, GloVe, ELMo, BERT, GPT.

> **Note**
>
> The three paradigms are not mutually exclusive. Modern systems often combine them

2.2 History of Word Embeddings

- 1950s: Distributional hypothesis (Harris 1954; Firth 1957)

> **Note**
>
> **Distributional models** (also called *distributional semantic models*) represent the meaning of a word as a function of the contexts in which it appears. They are grounded in the **distributional hypothesis** (Harris 1954; Firth 1957): *"a word is characterized by the company it keeps"* — words that occur in similar contexts tend to be related in meanings. These models derive meaning automatically from large corpora by counting co-occurrences.

- 1990: Latent Semantic Analysis (LSA) — Deerwester et al. (1990)
- 2003: Latent Dirichlet Allocation (LDA) — Blei et al. (2003)
- 2013: Word2Vec — Mikolov et al. (2013)
- 2014: GloVe — Pennington et al. (2014)

2.3 From Words to Vectors

Vector or distributional models of meaning are generally based on a co-occurrence matrix

2.3.1 Two common co-occurrence matrix

- the word-document matrix
- the word-word matrix.

2.3.2 Practical construction

- **tokenization** and **lemmatization** are common preprocessing steps to reduce sparsity and focus on content words.
  - a token is a unit of text (word, subword, character) that serves as the basic building block for vector representations.
  - Lemmatization reduces inflected forms to a common base form (lemma).
    - For example, "run", "runs", "running" would all be lemmatized to "run".
  - tokens like "the", "is", "and" are often removed as stop words.
- **The choice of context** (document, sentence, window) also affects the resulting vectors.

> **Note**
>
> In the document we will use `term`or `word` to refer to meaning units, and `context` to refer to the column labels. The context could be a document, a sentence, or a window of words around the **target word**.

2.4 Word-document matrix

In a Word-document matrix, each row represents a word in the vocabulary and each column represents a document from some collection of documents.

Corpus of 5 short sentences about NLP (used as a running example throughout this chapter):

Table 2.1: Term-document matrix — S1: *"word embeddings represent meaning"* · S2: *"deep neural networks learn representations"* · S3: *"language models process text"* · S4: *"word vectors capture context"* · S5: *"neural language models predict"*

	S1	S2	S3	S4	S5
word	1	0	0	1	0
embeddings	1	0	0	0	0
vectors	0	0	0	1	0
neural	0	1	0	0	1
language	0	0	1	0	1
models	0	0	1	0	1
context	0	0	0	1	0
meaning	1	0	0	0	0

2.5 Word-Word matrix

In the Word-context matrix the columns are labeled by words rather than documents.

- each cell records the number of times the row (target) word and the column (context) word co-occur in some context in some training corpus.

2.5.1 Context

- a context could be the document, in which case the cell represents the number of times the two words appear in the same document.
- smaller contexts are common: generally a window around the word

Word-word matrix for the same corpus (window $L = 1$, symmetric counts):

Table 2.2: Word-word co-occurrence matrix (window $L = 1$)

	word	embed-dings	vectors	represent	meaning	capture	context
word	—	1	1	0	0	0	0

	word	embed-dings	vectors	represent	meaning	capture	context
embeddings	1	—	0	1	0	0	0
vectors	1	0	—	0	0	1	0
represent	0	1	0	—	1	0	0
meaning	0	0	0	1	—	0	0
capture	0	0	1	0	0	—	1
context	0	0	0	0	0	1	—

> **Note**
>
> "embeddings" and "vectors" share the same context word ("word") → identical column profiles → cosine similarity = 1, even though they never appear in the same sentence.

2.6 Cosine for measuring similarity

The cosine similarity metric (or empirical correlation) between two word vectors v and w (lines of a word-document or word-word matrix)

$$\cos(v, w) = \frac{v^T w}{\|v\|\|w\|}$$

2.7 TF-IDF: Weigthing terms in the vector

2.7.1 Term frequency (TF)

- Raw frequency $TF_{t,d} = count(t, d)$ is very skewed and not very discriminative.
- Usually squashed by using the $log10$: a word appearing 100 times in a document doesn't make that word 100 times more relevant

$$\mathrm{TF}_{t,d} = \log_{10}(count(t, d) + 1)$$

2.7.2 Inverse document frequency

The second factor in tf-idf is used to give a higher weight to words that occur only in a few documents.

$$IDF_t = \log_{10}\left(\frac{N}{\mathrm{DF}_t}\right)$$

where N is the total number of documents, and DF_t is the number of documents in which term t occurs.

2.8 TF-IDF: Weigthing terms in the vector

The TF-IDF weighted value $w_{t,d}$ for term t in document d thus combines term frequency $TF_{t,d}$ with IDF_t:

$$w_{t,d} = TF_{t,d} \times IDF_t$$

Applied to our running corpus ($N = 5$ documents):

Table 2.3: TF-IDF weighted term-document matrix — TF = $\log_{10}(1 + 1) = 0.30$ for all present terms (each appears once per sentence)

	S1	S2	S3	S4	S5	idf
word	0.12	0	0	0.12	0	0.40
embeddings	**0.21**	0	0	0	0	0.70
vectors	0	0	0	**0.21**	0	0.70
neural	0	0.12	0	0	0.12	0.40
language	0	0	0.12	0	0.12	0.40
models	0	0	0.12	0	0.12	0.40
context	0	0	0	**0.21**	0	0.70
meaning	**0.21**	0	0	0	0	0.70

Words appearing in only one document ("embeddings", "vectors", "context", "meaning") receive the highest weight 0.21; words shared across two documents ("word", "neural", "language", "models") receive 0.12.

2.9 Pointwise Mutual Information (PMI)

An alternative weighting function to tf-idf, PPMI (positive pointwise mutual information), is used for **term(target)-term(context)-matrices**.

- Target word, word we focus on.
- Context words surrounding the target word

The pointwise mutual information between a **target word** w and a **context word** c (Church and Hanks 1989, Church and Hanks 1990) is defined as:

$$PMI(w, c) = \log_2 \frac{P(w, c)}{P(w)P(c)}$$

2.10 Positive PMI (called PPMI)

Negative PMI values (which imply things are co-occurring less often than we would expect by chance) tend to be unreliable unless our corpora are enormous.

2.10.1 Positive PMI

replaces all negative PMI values with zero (Church and Hanks 1989, Dagan et al. 1993, Niwa and Nitta 1994)

$$PPMI(w,c) = \max(\log_2 \frac{P(w,c)}{P(w)P(c)}, 0)$$

Warning

What if $P(w,c) = 0$?
When a pair (w,c) is never observed in the corpus:

$$P(w,c) = 0 \implies PMI(w,c) = \log_2 \frac{0}{P(w)P(c)} = -\infty$$

PPMI neutralises this *on the surface* — $\max(0, -\infty) = 0$ — but this hides an important ambiguity: after PPMI, a score of 0 can mean two very different things:

- the pair is **unobserved** (no information)
- the pair is **independent** (PMI exactly zero)

Common solutions

- **Smoothing**: add a small $\epsilon > 0$ to all counts before estimating probabilities, avoiding zeros at the cost of a slight bias.
- **Sparse storage**: store only non-zero entries — zeros are never computed. This is the most pragmatic and widespread approach in practice.

2.11 Example of PPMI weighting

Applied to our word-word matrix (16 context pairs total, window $L = 1$):

$$\text{PMI}(\text{embeddings}, \text{word}) = \log_2 \frac{P(\text{emb}, \text{word})}{P(\text{emb}) \cdot P(\text{word})} = \log_2 \frac{1/16}{(2/16)(2/16)} = \log_2 4 = 2.0$$

Table 2.4: PPMI matrix — all other co-occurrences have PMI ≤ 0, set to 0 by PPMI

	word	embeddings	vectors	represent	meaning
word	—	**2.0**	**2.0**	0	0
embeddings	**2.0**	—	0	**2.0**	0
vectors	**2.0**	0	—	0	0
represent	0	**2.0**	0	—	**2.0**
meaning	0	0	0	**2.0**	—

2.12 Applications of the TF-IDF or PPMI vector models

2.12.1 Principle

- Computing two documents (or word) similarity after transformation.
- Given two documents d_1 and d_2, the similarity is $\cos(d_1, d_2)$.

2.12.2 Applications

- **Documents** : information retrieval, plagiarism detection, news recommender systems, and even for digital humanities tasks like comparing different versions of a text to see which are similar to each other.
- **Words** : finding word paraphrases, tracking changes in word meaning, or automatically discovering meanings of words in different corpora.

2.13 Latent Semantic Analysis (LSA)

2.13.1 Goal and assumptions

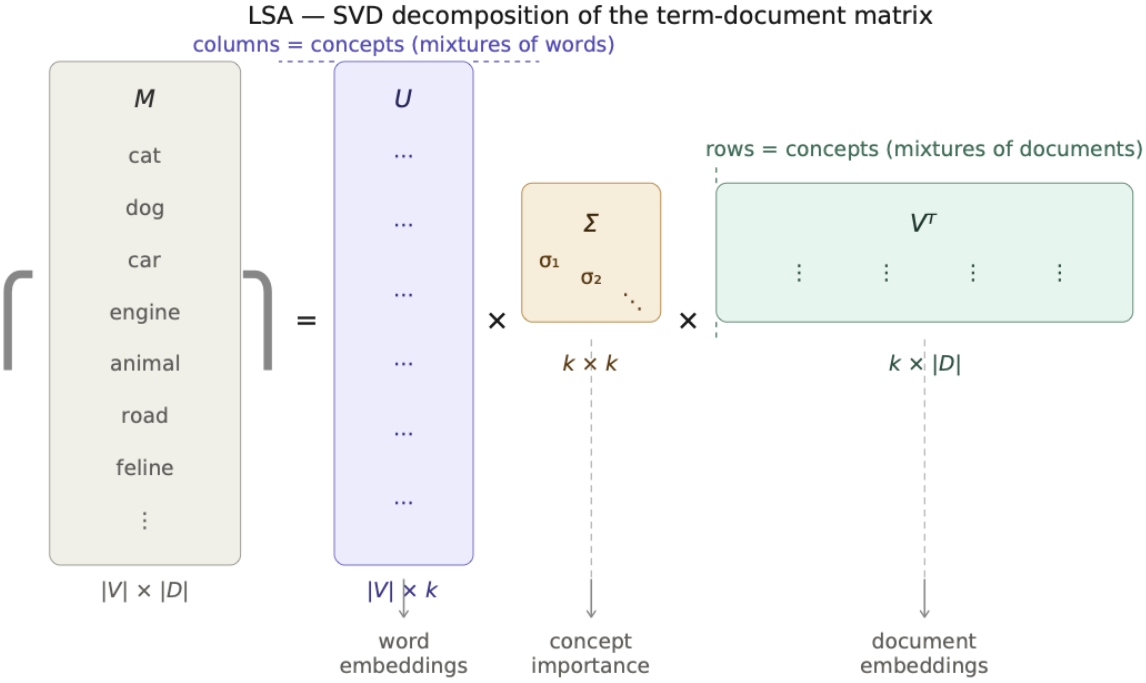
- extracting and representing the underlying meaning of words in a corpus of texts.
- words that occur in similar contexts have similar meanings.

2.13.2 Principle

LSA uses singular value decomposition (SVD) to identify latent, or hidden, patterns in word co-occurrence data.

2.14 Latent Semantic Analysis

LSA uses SVD on the matrix of word-document matrix to identify the latent concepts (contexts, topics, ...) that underlie the relationships between words in the corpus: $W = USV^T$ with classical low rank approximation $U_k S_k V_k^T$



2.15 Latent Semantic Analysis

- The values on the main diagonal of S_k indicate the 'importance' of each of the k main 'latent concepts' (or factors).
- For each of the document, the corresponding row of U_k allows us to see which concepts are present and with what weights.
- For each of the concept, the associated column of V_k indicates which terms form the concept (and with what weights).

Calculating the similarity between a term and a document involves choosing:

- the row $\mathbf{u}$ corresponding to the document in the matrix $\mathbf{U}_k$,
- the row $\mathbf{v}$ corresponding to the term in the matrix $\mathbf{V}_k$,
- and then computing the product $\mathbf{u}\mathbf{S}_k\mathbf{v}^T$.

To determine the similarity of a term to each of the documents

$$\mathbf{U}_k\mathbf{S}_k\mathbf{v}^T$$

Conversely, we can determine the similarities between a document $\mathbf{u}$ and all the terms through

$$\mathbf{u}\mathbf{S}_k\mathbf{V}_k^T$$

2.16 Latent Dirichlet Allocation (LDA)

2.16.1 History

In the context of population genetics, LDA was proposed by J. K. Pritchard, M. Stephens and P. Donnelly in 2000.

LDA was applied in machine learning by David Blei, Andrew Ng and Michael I. Jordan in 2003.

2.16.2 Principle

Latent Dirichlet Allocation (LDA) is a probabilistic generative model that assumes that

- each document is generated by a mixture of latent topics
- each topic is generated by a mixture of words from the vocabulary.

Documents are represented as random mixtures over latent topics, where each topic is characterized by a distribution over all the words.

2.17 LDA Generative process

For a corpus D consisting of M documents

1. Choose topic parameter vector for document i: $\theta_i \sim \mathrm{Dir}(\alpha)$, where $i \in \{1, \dots, M\}$ and $\mathrm{Dir}(\alpha)$ is a Dirichlet distribution with a symmetric parameter α which typically is sparse (α).

2. Choose the word parameters vector of topic k: $\varphi_k \sim \mathrm{Dir}(\beta)$, where $k \in \{1, \dots, K\}$ and β typically is sparse

3. For each of the word positions i, j, where $i \in \{1, \dots, M\}$, and $j \in \{1, \dots, N_i\}$

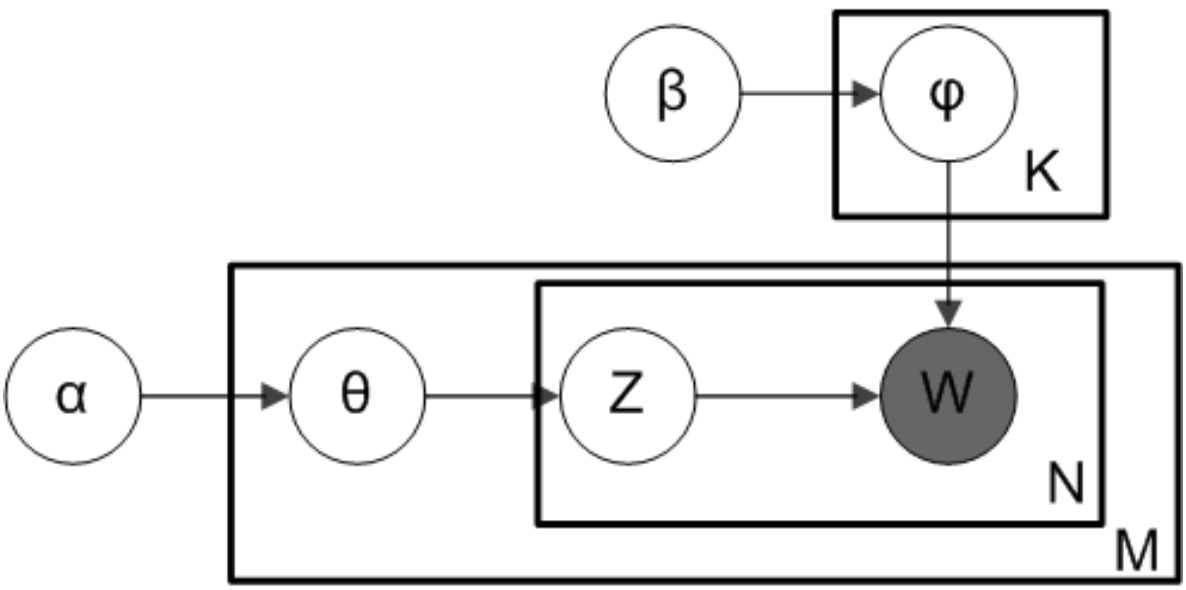
a. Choose a topic $z_{i,j} \sim \mathrm{Cat}(\theta_i)$.
b. Choose a word $w_{i,j} \sim \mathrm{Cat}(\varphi_{z_{i,j}})$.

2.18 Latent Dirichlet Allocation (LDA) in summary

$$\varphi_{k=1\ldots K} \sim \text{Dirichlet}_V(\beta) \tag{2.1}$$
$$\theta_{d=1\ldots M} \sim \text{Dirichlet}_K(\alpha) \tag{2.2}$$
$$z_{d=1\ldots M, w=1\ldots N_d} \sim \text{Categorical}_K(\theta_d) \tag{2.3}$$
$$w_{d=1\ldots M, w=1\ldots N_d} \sim \text{Categorical}_V(\varphi_{z_{dw}}) \tag{2.4}$$

$$P(W, Z, \theta, \varphi; \alpha, \beta) = \prod_{i=1}^{K} P(\varphi_i; \beta) \prod_{j=1}^{M} P(\theta_j; \alpha) \prod_{t=1}^{N} P(Z_{j,t} \mid \theta_j) P(W_{j,t} \mid \varphi_{Z_{j,t}}),$$

2.19 Latent Dirichlet Allocation (LDA) estimation



Estimating the parameters could be achieved through Variationnal Bayes EM algorithm.

2.20 Example of LDA in Action

Corpus of Three Documents

1. "cats dogs pets love"
2. "dogs bark loud outside"
3. "politics government election law"

Step 1: Identify Topics

After running LDA, it may discover two topics (dist. over words):

- Topic 1 (Pets): {cats, dogs, pets, bark, love}
- Topic 2 (Politics): {politics, government, election, law}

Step 2: Assign Topics to Documents

- Document 1 -> 90% Topic 1, 10% Topic 2
- Document 2 -> 80% Topic 1, 20% Topic 2
- Document 3 -> 95% Topic 2, 5% Topic 1

2.21 Pro and Cons

2.21.1 Advantages of LDA

- Unsupervised Learning: No labeled data needed.
- Interpretable Topics: Extracts meaningful topics from text.

2.21.2 Limitations of LDA

- Fixed Number of Topics: Must specify the number of topics beforehand.
- Bag-of-Words Assumption: Ignores word order and context.
- Computationally Expensive: Requires approximation methods for inference.

2.21.3 Applications of LDA

- Topic Modeling: Organizing large text corpora (news articles, research papers).
- Recommendation Systems: Suggesting articles based on topic similarity.
- Text Classification: Assigning labels based on topic distributions.
- Social Media Analysis: Identifying trending topics in posts.

2.22 Word2vec

2.22.1 Embeddings

- more powerful word representation
- short dense vectors: d ranging from 50-1000, rather than the much larger vocabulary size $|V|$ or number of documents D
- dense vectors work better in every NLP task than sparse vectors

2.22.2 Skip-gram with negative sampling

- The skip-gram algorithm is one of two algorithms in a software package called word2vec
- The other algorithm is CBOW (continuous bag of words)

The two algorithms share the same neural architecture but differ in the direction of prediction:

1. Skip-gram

- *Task*: given a target word w, predict its surrounding context words c.
- *Objective*: maximise $P(c \mid w)$ for each observed context word.
- Each occurrence of a word produces multiple training signals (one per context word in the window), making skip-gram particularly effective for **rare words**.
- Slower to train: generates many predictions per word.

2. CBOW (Continuous Bag of Words)

- *Task*: given a bag of context words $c_1, c_2, \ldots, c_k$, predict the central word w.
- *Objective*: maximise $P(w \mid c_1, c_2, \ldots, c_k)$.
- Context vectors are averaged before the prediction step, which smooths out context information.

- Faster to train: one prediction per window; works better for **frequent words**.

	Skip-gram	CBOW
Prediction direction	word → context	context → word
Training speed	Slower	Faster
Rare words	Better	Worse
Frequent words	Good	Better

2.23 Embedding derived from classification

2.23.1 Data : words and their context (neighboring words ($\pm L$)

word embeddings **represent** meaning

c1 c2 **w** c3

2.23.2 Classification task

Given a tuple $(w, c) \in \mathbb{R}^d \times R^d$ of a target word w paired with a candidate context word c (for example (apricot, jam), or perhaps (apricot, aardvark)) return the probability that c is a real context word (true for jam, false for aardvark):

$$P(y_{wc} = 1|w, c) = 1 - P(y_{wc} = 0|w, c)$$

2.23.3 Similarity

we rely on the intuition that two vectors are similar if they have a high dot product (cosine is just a normalized dot product):

$$Similarity(w, c) = c^T w$$

2.24 From dot product to logistic regression

2.24.1 Logit

$$P(y_{wc} = 1|w, c) = \frac{1}{1 + \exp(-c^T w)}$$

2.24.2 Criterion

The criterion is a kind of likelihood including positive examples (observed association) and negative examples (non observed associations). In fact skip-gram with negative sampling (SGNS) uses more negative examples than positive examples (with the ratio between them set by a parameter k).

$$L = \sum_{w,c:y_{wc}=1} \log P(y_{wc} = 1|w, c) + \sum_{w',c':y_{w'c'}=0} \log P(y_{w'c'} = 0|w', c')$$

2.25 Skip-gram training pairs

Sentence: *"word **embeddings** represent meaning"* — target = **embeddings**, window $L = 2$

Positive pairs (real context, $y = 1$):

target	context	label
embeddings	word	1
embeddings	represent	1
embeddings	meaning	1

Negative samples ($k = 2$ per positive, sampled $\propto P(w)^{3/4}$):

target	noise word	label
embeddings	neural	0
embeddings	deep	0
embeddings	process	0

The model is trained to push $c_{\text{word}}^T w_{\text{emb}} \to +\infty$ and $c_{\text{neural}}^T w_{\text{emb}} \to -\infty$.

After training on all sentences: $w_{\text{embeddings}} \approx w_{\text{vectors}}$ because both always appear next to "word".

2.26 Two-matrix architecture

Skip-gram maintains two $|V| \times d$ embedding matrices:

$$W = \begin{bmatrix} \leftarrow w_{\text{word}} \rightarrow \\ \leftarrow w_{\text{embeddings}} \rightarrow \\ \vdots \end{bmatrix} \qquad C = \begin{bmatrix} \leftarrow c_{\text{word}} \rightarrow \\ \leftarrow c_{\text{embeddings}} \rightarrow \\ \vdots \end{bmatrix}$$

- **Target matrix** W: used when the word is the focus of a training pair.
- **Context matrix** C: used when the word appears as context.

The logit for pair (w, c) is a single dot product: $c^T w$.

> **Note**
>
> The window size L controls the nature of the learned similarity: $L \leq 2$ captures syntactic relations · $L \geq 5$ captures topical/semantic relations.

2.27 Final embeddings

- skip-gram outputs the target matrix W and the context matrix C.
- It's common to just add them together, representing word i with the vector $w_i + c_i$.
- Alternatively we can throw away the C matrix and just represent each word i by the vector w_i.
- As with the simple count-based methods like tf-idf, the context window size L affects the performance of skip-gram embeddings, and experiments often tune the parameter L on a devset.

2.28 GloVe: from local windows to global statistics

Word2Vec learns from local context windows — each gradient step sees one (target, context) pair. The global co-occurrence structure of the corpus emerges only implicitly after many steps.

GloVe (Pennington et al., 2014) makes global statistics *explicit*:

1. Build the full co-occurrence matrix $\mathbf{X}$ where X_{ij} = number of times word j appears in the context of word i across the entire corpus.
2. Directly fit word vectors to this matrix.

> **Note**
>
> On our running corpus, $X_{\text{embeddings,word}} = 1$ and $X_{\text{vectors,word}} = 1$ — GloVe sees this symmetry globally and will assign similar vectors to both words.

2.29 GloVe objective

Learn $w_i, c_j \in \mathbb{R}^d$ (target and context vectors) and scalar biases b_i, b_j^c to minimise:

$$J = \sum_{i,j=1}^{|V|} f(X_{ij}) \left(w_i^T c_j + b_i + b_j^c - \log X_{ij}\right)^2$$

Weighting function — downweights rare pairs, caps frequent ones:

$$f(x) = \begin{cases} (x/x_{\max})^\alpha & x < x_{\max} \\ 1 & x \geq x_{\max} \end{cases} \qquad x_{\max} = 100,\ \alpha = 0.75$$

Intuition: the dot product of two word vectors should predict their log co-occurrence count: $w_i^T c_j \approx \log X_{ij}$.

2.30 Interpretation of the GloVe objective

The decomposition of the joint log probability is:

$$\log P(w_1, w_2) = \underbrace{\log P(w_2 \mid w_1)}_{\text{relational signal}} + \underbrace{\log P(w_1)}_{\text{marginal of } w_1 \text{ only}}$$

If the dot product modelled $\log P(w_1, w_2)$ directly, it would encode the **marginal frequency** of each word — information irrelevant to the *relationship* between the pair.

Instead, GloVe decomposes the target explicitly:

$$\underbrace{w_i^T c_j}_{\text{relational signal}} + \underbrace{b_i + b_j^c}_{\text{absorb marginals}} \approx \log X_{ij}$$

The bias terms soak up $\log P(w_i)$ and $\log P(w_j)$, so the dot product converges to:

$$w_i^T c_j \approx \log P(w_j \mid w_i) \approx \mathrm{PMI}(i,j) + \mathrm{const}$$

> **Note**
>
> **PMI is the natural target for a dot product**: $\mathrm{PMI}(w,c) = \log P(w,c) - \log P(w) - \log P(c)$ strips both marginals, leaving pure relational signal. Levy & Goldberg (2014) showed that Word2Vec SGNS implicitly factorises the **shifted PMI matrix**.

2.31 GloVe: meaning from co-occurrence ratios

The key insight is that **ratios** of co-occurrence probabilities encode semantic relations:

	$P(k \mid \text{ice})$	$P(k \mid \text{steam})$	ratio
$k = \text{solid}$	large	small	$\gg 1$
$k = \text{gas}$	small	large	$\ll 1$
$k = \text{water}$	large	large	≈ 1
$k = \text{fashion}$	small	small	≈ 1

GloVe vectors encode these ratios directly:

$$w_i^T c_k - w_j^T c_k \approx \log \frac{P(k \mid i)}{P(k \mid j)}$$

This explains why linear analogy arithmetic ($w_{\text{king}} - w_{\text{man}} + w_{\text{woman}} \approx w_{\text{queen}}$) works: the vector difference encodes a log probability ratio.

2.32 Word2Vec vs GloVe

	Word2Vec (SGNS)	GloVe
Training signal	Local windows, one pair at a time	Global co-occurrence matrix $\mathbf{X}$
Objective	Logistic classification	Weighted least squares
Scalability	Streams corpus (SGD)	Requires building $\mathbf{X}$ first
Small corpora	Better (no need to store $\mathbf{X}$)	Worse
Interpretability	Implicit	Explicit ($\log X_{ij}$ target)
Analogy accuracy	High	High (comparable)

Pre-trained vectors are available for both: `word2vec-google-news-300` (100B tokens), `glove-wiki-gigaword-{50,100,200,300}d` (6B tokens).

! Important

Shared limitation: both methods produce **one static vector per word** regardless of context. The word *bank* gets the same representation whether it means a riverbank or a financial institution. Contextualised embeddings — ELMo, BERT, GPT — overcome this by making representations depend on the full sentence context.

2.33 Analogy Tasks

2.33.1 The Core Idea

Mikolov et al. (2013) observed that semantic relationships are encoded as **linear translations** in the embedding space:

$$\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$$

The vector **difference** captures a relation, and that relation is approximately **consistent** across all pairs sharing the same relation.

2.33.2 Solving an Analogy

Given the query *"man is to king as woman is to ?"*, compute:

$$\hat{v} = \vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}}$$

Then find the word w^* whose embedding is closest to $\hat{v}$:

$$w^* = \arg \max_{w \notin \{\text{man, king, woman}\}} \cos(\vec{w},\ \hat{v})$$

The query words are excluded to prevent the trivial answer (e.g. returning "king" itself).

2.33.3 Why It Works

If the model has learned a consistent **gender direction** δ_g:

$$\vec{\text{king}} - \vec{\text{queen}} \approx \vec{\text{man}} - \vec{\text{woman}} \approx \delta_g$$

then the embedding space organises words so that a given *relation* corresponds to a *constant direction*. The GloVe objective makes this explicit: since $w_i^T c_k \approx \log P(k \mid i)$, the vector difference encodes a log co-occurrence ratio:

$$w_{\text{king}}^T c_k - w_{\text{queen}}^T c_k \approx \log \frac{P(k \mid \text{king})}{P(k \mid \text{queen})}$$

2.33.4 Standard Benchmark

The **Google Analogy Dataset** (Mikolov 2013) contains ~19 500 questions in two categories:

Category	Example
Semantic	Paris : France :: Berlin : ? *(Germany)*
Semantic	king : man :: queen : ? *(woman)*
Syntactic	run : running :: swim : ? *(swimming)*
Syntactic	good : better :: bad : ? *(worse)*

Accuracy = fraction of questions where the top-1 nearest neighbour is exactly the expected word.

> **Note**
>
> Both Word2Vec and GloVe achieve comparable accuracy on this benchmark. The linearity assumption is approximate — it breaks for irregular forms (*good* → *best*, not *gooder*) and for rare words with noisy embeddings.

Chapter 3

Exercises — LSA and Word2Vec Embeddings

These exercises explore two complementary approaches to building word representations:

- **Latent Semantic Analysis (LSA)** — a classical count-based method using SVD on a term-document matrix
- **Word2Vec** — a neural prediction-based method using skip-gram with negative sampling

Both methods were introduced in the lecture (`word-embedding.qmd`). The practical work is in `TP/lsa_word2vec.ipynb`.

3.1 Links

- Original LSA paper — Deerwester et al. (1990)
- Word2Vec paper — Mikolov et al. (2013)
- GloVe paper — Pennington et al. (2014)
- Gensim documentation — Word2Vec and pre-trained vectors
- Jurafsky & Martin, Chapter 6 — Vector semantics and embeddings

---

3.1.1 1) Term-document matrix and cosine similarity

Goal: Build a count matrix from a toy corpus and measure word similarity by cosine distance.

Key ideas

- A corpus of D documents with a vocabulary of $|V|$ words defines a $|V| \times D$ term-document matrix.
- Raw counts can be weighted (TF-IDF) to reduce the influence of frequent uninformative words.
- Cosine similarity $\cos(\mathbf{v}, \mathbf{w}) = \frac{\mathbf{v}^T\mathbf{w}}{\|\mathbf{v}\|\|\mathbf{w}\|}$ is the standard metric between word vectors.

Exercises

1. Build the term-document matrix with `CountVectorizer`. What is its shape? How sparse is it?
2. Switch to `TfidfVectorizer`. Which words gain weight? Which lose weight?
3. Compute cosine similarity between pairs of words from the same topic and across topics. Do the numbers match your intuition?

3.1.2 2) Latent Semantic Analysis — SVD and latent space

Goal: Apply Truncated SVD to reduce dimensionality and visualize the latent semantic structure.

Key ideas

- LSA factorizes the matrix $W = U\Sigma V^T$ and keeps only the top k singular values: $\hat{W} = U_k \Sigma_k V_k^T$.
- The rows of V_k are document embeddings; the rows of U_k (scaled by Σ_k) are word embeddings.
- Dimensionality reduction acts as denoising: synonymous words that never co-occur in the same document become close in the latent space.

Exercises

1. Apply `TruncatedSVD` with $k = 2$ and plot documents in the latent plane. Do the three topics separate cleanly?
2. Plot the word vectors in the same plane. Which words anchor each dimension?
3. Increase k to 5 and 10. How does explained variance change? Use the document embeddings to retrieve the top-3 documents for a free-text query.
4. **Reflection:** What does each singular value measure geometrically? Why does LSA struggle with polysemy (e.g. *star* = celebrity or astronomical object)?

3.1.3 3) Word2Vec on the toy corpus

Goal: Train a skip-gram Word2Vec model and compare its embeddings with LSA.

Key ideas

- Word2Vec trains a binary classifier: given a (target, context) pair, predict whether the context is real or noise-sampled.
- The objective $L = \sum_{(w,c):y=1} \log \sigma(\mathbf{c}^T\mathbf{w}) + \sum_{(w',c'):y=0} \log \sigma(-\mathbf{c}'^T\mathbf{w}')$ is maximized by gradient descent.
- The context window L controls what "context" means: small L captures syntactic relations, large L captures topical/semantic relations.

Exercises

1. Train Word2Vec (`sg=1`, `window=3`, `epochs=200`) on the toy corpus. Visualize the embeddings with PCA.
2. Compare side-by-side: for the same word pairs, are Word2Vec similarities higher/lower/more discriminative than LSA?
3. Try `sg=0` (CBOW) and `window=1` vs `window=5`. What changes in the clustering?
4. **Reflection:** Why does Word2Vec underperform on this 18-sentence corpus? What data regime favours LSA?

3.1.4 4) Pre-trained embeddings and analogy tasks

Goal: Load large-scale GloVe vectors and test the famous linear structure of word embeddings.

Key ideas

- Pre-trained GloVe vectors are trained on Wikipedia + Gigaword (~6 billion tokens, 400k vocabulary, 50–300 dimensions).
- The linear analogy structure **king** − **man** + **woman** ≈ **queen** emerges from training at scale.
- This structure does **not** appear in LSA or Word2Vec trained on small corpora.

Exercises

1. Load `glove-wiki-gigaword-50` with `gensim.downloader`. Solve the analogies: *king − man + woman*, *Paris − France + Italy*, *walk − walking + swimming*.
2. Try to solve analogies with your toy LSA vectors. What happens?
3. Find 3 word pairs where GloVe similarities agree with LSA, and 3 where they strongly disagree. What explains the disagreements?

3.1.5 5) Comparison and reflection

Goal: Synthesise the trade-offs between LSA and Word2Vec.

Property	LSA	Word2Vec
Method	SVD on count matrix	Neural skip-gram
Training	Closed-form	Gradient descent
Corpus size needed	Small OK	Needs billions of words
Polysemy handling	Poor (single vector)	Poor (single vector)
Analogy structure	Weak	Strong at scale
Interpretability	Moderate (SVD components)	Low
Matrix storage	$O(\|V\| \cdot D)$	$O(\|V\| \cdot d)$

Exercises

1. In which practical scenario would you choose LSA over Word2Vec? Give two concrete examples.
2. Both methods produce a **single vector per word** regardless of context. Why is this a fundamental limitation? What architecture overcomes it? (Hint: see `chapters/transformers.qmd`.)
3. The lecture notes describe PPMI as an alternative to raw counts before SVD. Implement PPMI weighting and compare LSA-PPMI similarity with raw-count LSA and Word2Vec.

Chapter 4

Embeddings

This notebook explores two key approaches to word representation:

1. **Part 1 — Latent Semantic Analysis (LSA):** classical count-based method using SVD on a term-document matrix.
2. **Part 2 — Word2Vec:** neural prediction-based method using skip-gram with negative sampling.
3. **Part 3 — Comparison:** side-by-side analysis of both methods, including pre-trained GloVe vectors.

Dependencies: `numpy`, `matplotlib`, `scikit-learn`, `gensim`

```
pip install numpy matplotlib scikit-learn gensim
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.decomposition import TruncatedSVD, PCA
from sklearn.metrics.pairwise import cosine_similarity
import warnings
warnings.filterwarnings('ignore')

try:
    from gensim.models import Word2Vec
    import gensim.downloader
    print("gensim available ")
except ImportError:
    print("Install gensim: pip install gensim")
```

```
gensim available
```

4.1 Part 1 — Latent Semantic Analysis (LSA)

LSA applies **Singular Value Decomposition (SVD)** to a term-document matrix to discover latent semantic structure:

$$W = U\Sigma V^T \approx U_k \Sigma_k V_k^T$$

Matrix	Shape	Meaning
U_k	$\|V\| \times k$	Word representations in the latent space
V_k	$D \times k$	Document representations in the latent space
Σ_k	$k \times k$	Importance of each latent dimension

4.1.1 1.1 — Toy corpus

```
# Three-topic toy corpus: Space, Sports, Cooking
corpus = [
    # Space (docs 0-5)
    "The sun is a star at the center of the solar system",
    "Stars are made of hydrogen and helium gas",
    "Planets orbit around the sun in elliptical paths",
    "The moon orbits the earth and affects the tides",
    "Telescopes allow astronomers to observe distant galaxies",
    "Black holes are regions where gravity is extremely strong",
    # Sports (docs 6-11)
    "Football is a popular sport played with a round ball",
    "Players kick the ball to score goals in the net",
    "The team won the championship after training hard",
    "Athletes run and jump during competitions and matches",
    "The goalkeeper saved the shot and won the match",
    "Basketball players dribble and shoot the ball",
    # Cooking (docs 12-17)
    "The chef cooks pasta with tomato sauce and herbs",
    "Recipes require fresh ingredients like vegetables and spices",
    "Baking bread needs flour water yeast and salt",
    "The kitchen smells of garlic and fresh herbs",
    "Vegetables and fruits are healthy food for the body",
    "Cooking involves heat to transform raw ingredients",
]

doc_labels = ["Space"] * 6 + ["Sports"] * 6 + ["Cooking"] * 6
topic_colors = {"Space": "steelblue", "Sports": "tomato", "Cooking": "seagreen"}

print(f"Corpus: {len(corpus)} documents across 3 topics")
for i, doc in enumerate(corpus):
    print(f"  [{doc_labels[i]:7s}] {doc}")
```

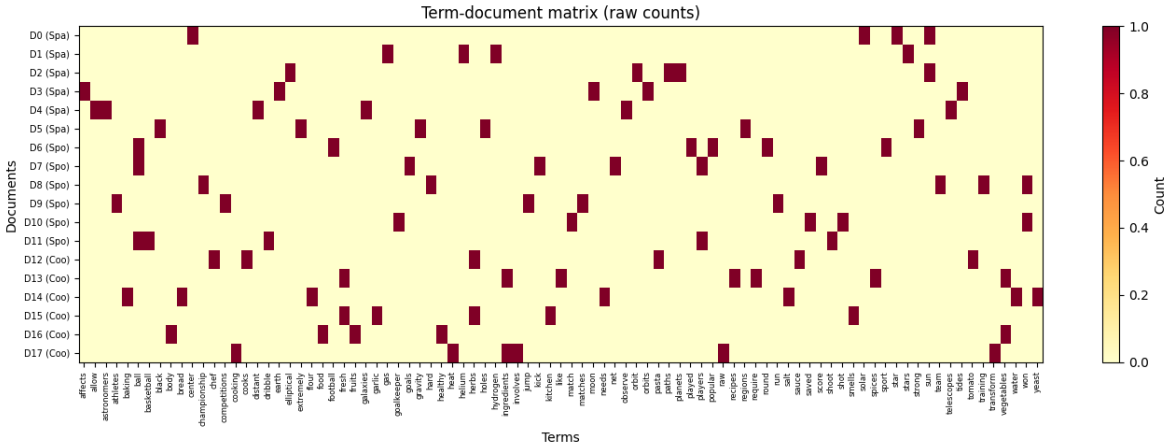
```
Corpus: 18 documents across 3 topics
  [Space  ] The sun is a star at the center of the solar system
  [Space  ] Stars are made of hydrogen and helium gas
  [Space  ] Planets orbit around the sun in elliptical paths
  [Space  ] The moon orbits the earth and affects the tides
  [Space  ] Telescopes allow astronomers to observe distant galaxies
  [Space  ] Black holes are regions where gravity is extremely strong
  [Sports ] Football is a popular sport played with a round ball
  [Sports ] Players kick the ball to score goals in the net
  [Sports ] The team won the championship after training hard
```

```
[Sports ] Athletes run and jump during competitions and matches
[Sports ] The goalkeeper saved the shot and won the match
[Sports ] Basketball players dribble and shoot the ball
[Cooking] The chef cooks pasta with tomato sauce and herbs
[Cooking] Recipes require fresh ingredients like vegetables and spices
[Cooking] Baking bread needs flour water yeast and salt
[Cooking] The kitchen smells of garlic and fresh herbs
[Cooking] Vegetables and fruits are healthy food for the body
[Cooking] Cooking involves heat to transform raw ingredients
```

```python
# Build term-document matrix (documents × terms)
vectorizer = CountVectorizer(stop_words='english', min_df=1)
X = vectorizer.fit_transform(corpus)  # sparse matrix (n_docs, n_terms)
vocab = vectorizer.get_feature_names_out()

print(f"Vocabulary size : {len(vocab)} terms")
print(f"Matrix shape    : {X.shape}  (documents × terms)")
print(f"Sparsity        : {1 - X.nnz / (X.shape[0] * X.shape[1]):.1%} zeros")
print(f"\nFirst 30 terms  : {list(vocab[:30])}")
```

```
Vocabulary size : 89 terms
Matrix shape    : (18, 89)  (documents × terms)
Sparsity        : 93.9% zeros

First 30 terms  : ['affects', 'allow', 'astronomers', 'athletes', 'baking', 'ball', 'basketball', 'bl
```

```python
# Visualise the term-document matrix as a heatmap
fig, ax = plt.subplots(figsize=(14, 5))
im = ax.imshow(X.toarray(), aspect='auto', cmap='YlOrRd')
ax.set_xlabel("Terms")
ax.set_ylabel("Documents")
ax.set_yticks(range(len(corpus)))
ax.set_yticklabels([f"D{i} ({doc_labels[i][:3]})" for i in range(len(corpus))], fontsize=7)
ax.set_xticks(range(len(vocab)))
ax.set_xticklabels(vocab, rotation=90, fontsize=6)
plt.colorbar(im, ax=ax, label="Count")
ax.set_title("Term-document matrix (raw counts)")
plt.tight_layout()
plt.show()
```



Exercise 1.1 — Which columns (terms) have non-zero values in multiple topic rows? What does that mean for raw cosine similarity between those topic documents?

```
# Exercise 1.1 - terms shared across topics + impact on raw cosine similarity

# Binary term presence by document
X_bin = (X > 0).astype(int)

# Term presence by topic
topics = sorted(set(doc_labels))
topic_term_presence = {}
for t in topics:
    doc_idx = [i for i, lbl in enumerate(doc_labels) if lbl == t]
    topic_term_presence[t] = np.asarray(X_bin[doc_idx].sum(axis=0)).ravel() > 0

# Terms appearing in at least 2 topics
shared_mask = np.sum(np.vstack([topic_term_presence[t] for t in topics]), axis=0) >= 2
shared_terms = vocab[shared_mask]

print(f"Terms present in multiple topics ({len(shared_terms)}):")
print(shared_terms.tolist())

# Raw cosine similarity between all documents (count space)
S = cosine_similarity(X)

same_topic_sims, cross_topic_sims = [], []
n_docs = len(doc_labels)
for i in range(n_docs):
    for j in range(i + 1, n_docs):
        if doc_labels[i] == doc_labels[j]:
            same_topic_sims.append(S[i, j])
        else:
            cross_topic_sims.append(S[i, j])

print("\nAverage cosine similarity (raw counts):")
print(f"  Same-topic  : {np.mean(same_topic_sims):.3f}")
```

```
print(f"  Cross-topic : {np.mean(cross_topic_sims):.3f}")

print("\nInterpretation:")
print("- Shared terms create non-zero overlap between different topics.")
print("- This increases cross-topic cosine similarity, making topic separation less clean in raw coun
```

```
Terms present in multiple topics (0):
[]

Average cosine similarity (raw counts):
  Same-topic  : 0.040
  Cross-topic : 0.000

Interpretation:
- Shared terms create non-zero overlap between different topics.
- This increases cross-topic cosine similarity, making topic separation less clean in raw count space
```

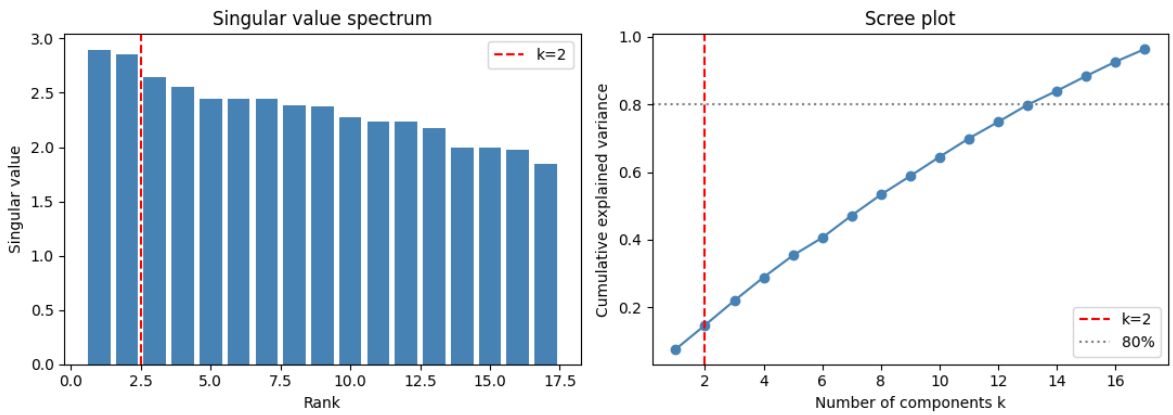
4.1.3 1.3 — Singular Value Decomposition

```
k = 2  # number of latent dimensions
svd = TruncatedSVD(n_components=k, random_state=42)

# X_docs: document embeddings (n_docs, k)
X_docs = svd.fit_transform(X)

# Word embeddings: rows of V^T scaled by singular values
# svd.components_ is (k, n_terms), so transpose → (n_terms, k)
X_words = svd.components_.T  # (n_terms, k)

print(f"Singular values : {svd.singular_values_.round(2)}")
print(f"Explained variance: dim1={svd.explained_variance_ratio_[0]:.1%}, "
      f"dim2={svd.explained_variance_ratio_[1]:.1%}, "
      f"total={svd.explained_variance_ratio_.sum():.1%}")
print(f"\nDocument embedding shape : {X_docs.shape}")
print(f"Word embedding shape     : {X_words.shape}")
```

```
Singular values : [2.9  2.85]
Explained variance: dim1=7.7%, dim2=7.1%, total=14.8%

Document embedding shape : (18, 2)
Word embedding shape     : (89, 2)
```

```
# Plot singular value spectrum
svd_full = TruncatedSVD(n_components=min(17, len(vocab)-1), random_state=42)
svd_full.fit(X)

fig, axes = plt.subplots(1, 2, figsize=(11, 4))

axes[0].bar(range(1, len(svd_full.singular_values_)+1), svd_full.singular_values_, color='steelblue')
axes[0].set_xlabel("Rank")
axes[0].set_ylabel("Singular value")
```

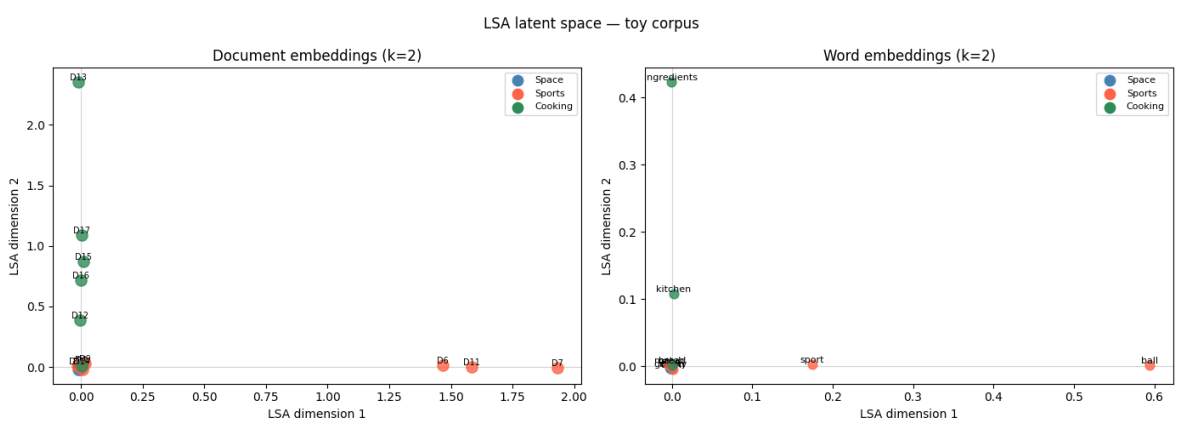
```
axes[0].set_title("Singular value spectrum")
axes[0].axvline(k + 0.5, color='red', linestyle='--', label=f"k={k}")
axes[0].legend()

cumvar = np.cumsum(svd_full.explained_variance_ratio_)
axes[1].plot(range(1, len(cumvar)+1), cumvar, marker='o', color='steelblue')
axes[1].axvline(k, color='red', linestyle='--', label=f"k={k}")
axes[1].axhline(0.8, color='gray', linestyle=':', label="80%")
axes[1].set_xlabel("Number of components k")
axes[1].set_ylabel("Cumulative explained variance")
axes[1].set_title("Scree plot")
axes[1].legend()

plt.tight_layout()
plt.show()
```



4.1.4 1.4 — Visualising the latent space

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# --- Document plot ---
ax = axes[0]
for i, (x, y) in enumerate(X_docs):
    color = topic_colors[doc_labels[i]]
    ax.scatter(x, y, color=color, s=90, alpha=0.8, zorder=3)
    ax.annotate(f"D{i}", (x, y), fontsize=7, ha='center', va='bottom')
for label, color in topic_colors.items():
    ax.scatter([], [], color=color, label=label, s=80)
ax.legend(fontsize=8)
ax.axhline(0, color='lightgray', lw=0.8)
ax.axvline(0, color='lightgray', lw=0.8)
ax.set_xlabel("LSA dimension 1")
ax.set_ylabel("LSA dimension 2")
ax.set_title("Document embeddings (k=2)")

# --- Word plot ---
```

```
target_words = [
    "sun", "star", "moon", "orbit", "telescope", "gravity",
    "ball", "team", "player", "goal", "sport", "match",
    "cook", "recipe", "bread", "vegetable", "kitchen", "ingredients",
]
word_topic = (
    ["Space"] * 6 + ["Sports"] * 6 + ["Cooking"] * 6
)
ax = axes[1]
for word, topic in zip(target_words, word_topic):
    if word in vocab:
        idx = np.where(vocab == word)[0][0]
        x, y = X_words[idx]
        ax.scatter(x, y, color=topic_colors[topic], s=60, alpha=0.8, zorder=3)
        ax.annotate(word, (x, y), fontsize=8, ha='center', va='bottom')
for label, color in topic_colors.items():
    ax.scatter([], [], color=color, label=label, s=80)
ax.legend(fontsize=8)
ax.axhline(0, color='lightgray', lw=0.8)
ax.axvline(0, color='lightgray', lw=0.8)
ax.set_xlabel("LSA dimension 1")
ax.set_ylabel("LSA dimension 2")
ax.set_title("Word embeddings (k=2)")

plt.suptitle("LSA latent space - toy corpus", fontsize=12)
plt.tight_layout()
plt.show()
```

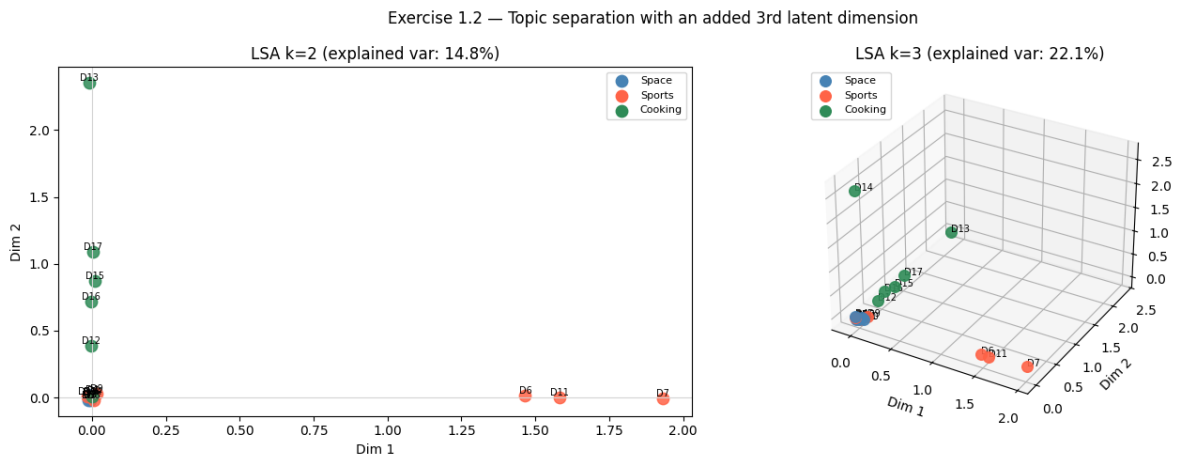


Exercise 1.2 — Which dimension separates which topics? Try `k=3`: add a third plot axis. Does the separation improve?

Exercise 1.3 — Replace `CountVectorizer` with `TfidfVectorizer` (same `stop_words='english'`). Re-run SVD and compare the document plots. Do the topics separate more cleanly?

```
# Exercise 1.2 - Compare k=2 vs k=3 and visualize the added latent axis
svd3 = TruncatedSVD(n_components=3, random_state=42)
X_docs_3 = svd3.fit_transform(X)
```

```
fig = plt.figure(figsize=(14, 5))

# k=2 (existing)
ax1 = fig.add_subplot(1, 2, 1)
for i, (x, y) in enumerate(X_docs):
    c = topic_colors[doc_labels[i]]
    ax1.scatter(x, y, color=c, s=80, alpha=0.85)
    ax1.annotate(f"D{i}", (x, y), fontsize=7, ha="center", va="bottom")
for label, c in topic_colors.items():
    ax1.scatter([], [], color=c, s=80, label=label)
ax1.legend(fontsize=8)
ax1.axhline(0, color="lightgray", lw=0.8)
ax1.axvline(0, color="lightgray", lw=0.8)
ax1.set_xlabel("Dim 1")
ax1.set_ylabel("Dim 2")
ax1.set_title(f"LSA k=2 (explained var: {svd.explained_variance_ratio_.sum():.1%})")

# k=3 (new axis)
ax2 = fig.add_subplot(1, 2, 2, projection="3d")
for i, (x, y, z) in enumerate(X_docs_3):
    c = topic_colors[doc_labels[i]]
    ax2.scatter(x, y, z, color=c, s=70, alpha=0.9)
    ax2.text(x, y, z, f"D{i}", fontsize=7)
for label, c in topic_colors.items():
    ax2.scatter([], [], [], color=c, s=70, label=label)
ax2.legend(fontsize=8, loc="best")
ax2.set_xlabel("Dim 1")
ax2.set_ylabel("Dim 2")
ax2.set_zlabel("Dim 3")
ax2.set_title(f"LSA k=3 (explained var: {svd3.explained_variance_ratio_.sum():.1%})")

plt.suptitle("Exercise 1.2 - Topic separation with an added 3rd latent dimension", fontsize=12)
plt.tight_layout()
plt.show()

# Quick diagnostic: which dimension separates topic centroids most
labels_arr = np.array(doc_labels)
topic_means = np.vstack([X_docs_3[labels_arr == t].mean(axis=0) for t in sorted(set(doc_labels))])
sep_strength = topic_means.var(axis=0)  # higher => stronger topic separation across centroids
print("Centroid-separation strength by dimension (k=3):")
for d, v in enumerate(sep_strength, start=1):
    print(f"  Dim {d}: {v:.4f}")
print(f"Most separating dimension: Dim {np.argmax(sep_strength) + 1}")
```




```
Centroid-separation strength by dimension (k=3):
  Dim 1: 0.1628
  Dim 2: 0.1758
  Dim 3: 0.0442
Most separating dimension: Dim 2
```

4.1.5 1.5 — Word similarity and document retrieval

```python
def lsa_word_sim(w1, w2):
    """Cosine similarity between two words in LSA space."""
    if w1 not in vocab or w2 not in vocab:
        return None
    v1 = X_words[np.where(vocab == w1)[0][0]] # get the row corresponding to w1
    v2 = X_words[np.where(vocab == w2)[0][0]] # get the row corresponding to w2
    return float(cosine_similarity([v1], [v2])[0, 0])

same_topic_pairs  = [("sun", "moon"), ("ball", "team"), ("cook", "recipe")]
cross_topic_pairs = [("sun", "ball"), ("moon", "cook"), ("star", "goal")]

print("LSA Word Similarities")
print(f"{'Pair':<25} {'Similarity':>10}")
print("-" * 37)
print("Same-topic pairs (expected: high)")
for w1, w2 in same_topic_pairs:
    sim = lsa_word_sim(w1, w2)
    tag = " " if sim and sim > 0.5 else " "
    print(f"  {w1}  {w2:<18} {sim:>8.3f}  {tag}" if sim else f"  {w1}  {w2}: N/A")
print("Cross-topic pairs (expected: low)")
for w1, w2 in cross_topic_pairs:
    sim = lsa_word_sim(w1, w2)
    tag = " " if sim and abs(sim) < 0.3 else " "
    print(f"  {w1}  {w2:<18} {sim:>8.3f}  {tag}" if sim else f"  {w1}  {w2}: N/A")
```

```
LSA Word Similarities
Pair                      Similarity
```

```
------------------------------------
Same-topic pairs (expected: high)
  sun   moon                  0.800
  ball   team                  0.383
  cook   recipe: N/A
Cross-topic pairs (expected: low)
  sun   ball                 -0.813
  moon   cook: N/A
  star   goal: N/A
```

```python
def lsa_query(query_text, top_n=3):
    """Retrieve top_n most similar documents to a free-text query."""
    q_vec = vectorizer.transform([query_text])
    q_lsa = svd.transform(q_vec)
    sims   = cosine_similarity(q_lsa, X_docs)[0]
    top    = sims.argsort()[::-1][:top_n]
    print(f"Query: '{query_text}'")
    for i in top:
        print(f"  [{sims[i]:.3f}] ({doc_labels[i]:7s}) {corpus[i]}")
    print()

lsa_query("orbit around the sun")
lsa_query("score goals with a ball")
lsa_query("fresh ingredients in the kitchen")
```

```
Query: 'orbit around the sun'
  [0.998] (Space  ) Black holes are regions where gravity is extremely strong
  [0.979] (Space  ) Planets orbit around the sun in elliptical paths
  [0.946] (Space  ) The moon orbits the earth and affects the tides

Query: 'score goals with a ball'
  [1.000] (Sports ) Players kick the ball to score goals in the net
  [1.000] (Sports ) Basketball players dribble and shoot the ball
  [1.000] (Sports ) Football is a popular sport played with a round ball

Query: 'fresh ingredients in the kitchen'
  [1.000] (Cooking) Cooking involves heat to transform raw ingredients
  [1.000] (Cooking) Vegetables and fruits are healthy food for the body
  [1.000] (Cooking) Recipes require fresh ingredients like vegetables and spices
```

Exercise 1.4 — Query with the ambiguous sentence `"the star player scored a goal"`. Which topic does LSA retrieve? Why? What does this reveal about how LSA handles polysemy?

Exercise 1.5 (bonus) — Implement PPMI weighting before SVD:

$$\mathrm{PPMI}(w, c) = \max\left(\log_2 \frac{P(w, c)}{P(w)P(c)}, 0\right)$$

Compare the resulting word similarities with raw counts.

4.2 Part 2 — Word2Vec

Word2Vec trains a **skip-gram classifier**: given a target word $\mathbf{w}$, predict which words appear in its context window $\pm L$.

$$L = \sum_{(w,c)\,:\,y=1} \log \sigma(\mathbf{c}^T \mathbf{w}) + \sum_{(w',c')\,:\,y=0} \log \sigma(-\mathbf{c}'^T \mathbf{w}')$$

Unlike LSA, Word2Vec is a **predictive** model optimised by gradient descent rather than matrix factorisation.

4.2.1 2.1 — Training on the toy corpus

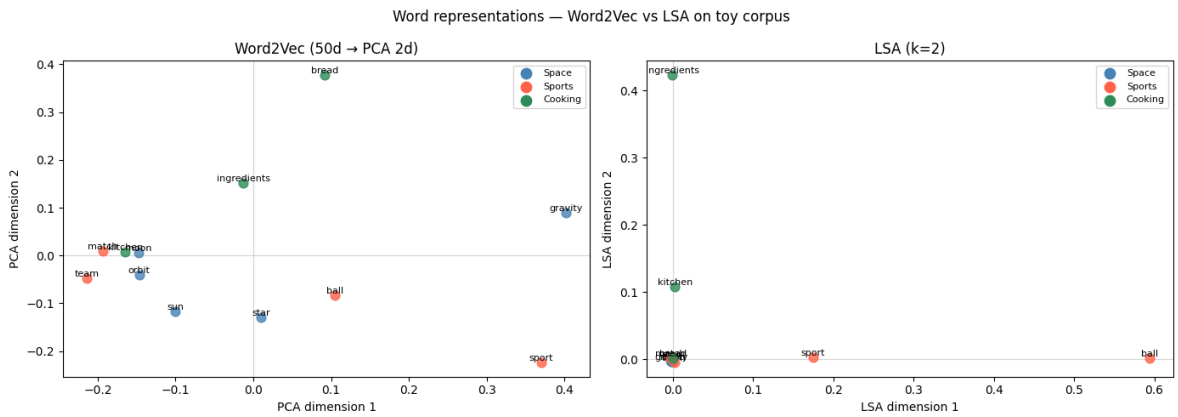
```
# Tokenise corpus (lower-case, split on spaces)
tokenized = [doc.lower().split() for doc in corpus]

# Train Word2Vec - skip-gram (sg=1), window ±3
w2v = Word2Vec(
    sentences=tokenized,
    vector_size=50,   # embedding dimension d
    window=3,         # context window ±3
    min_count=1,      # keep all words
    sg=1,             # 1 = skip-gram, 0 = CBOW
    negative=5,       # negative samples per positive
    epochs=300,       # more epochs for small corpus
    seed=42,
    workers=1,        # deterministic with seed
)

print(f"Vocabulary size   : {len(w2v.wv)} words")
print(f"Embedding dimension: {w2v.vector_size}")
print(f"\nClosest words to 'sun':")
for word, sim in w2v.wv.most_similar("sun", topn=5):
    print(f"  {word:<15} {sim:.3f}")
```

```
Vocabulary size   : 106 words
Embedding dimension: 50

Closest words to 'sun':
  planets         0.990
  around          0.989
  elliptical      0.988
  orbit           0.987
  paths           0.985
```

4.2.2 2.2 — Visualising Word2Vec embeddings

```
# Project 50-d vectors to 2D with PCA for visualisation
words_to_plot = [w for w in target_words if w in w2v.wv]
w2v_vecs = np.array([w2v.wv[w] for w in words_to_plot])
```

```
pca = PCA(n_components=2, random_state=42)
w2v_2d = pca.fit_transform(w2v_vecs)

word_to_topic = dict(zip(
    target_words,
    ["Space"]*6 + ["Sports"]*6 + ["Cooking"]*6
))

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# --- Word2Vec plot ---
ax = axes[0]
for word, (x, y) in zip(words_to_plot, w2v_2d):
    color = topic_colors[word_to_topic[word]]
    ax.scatter(x, y, color=color, s=60, alpha=0.8, zorder=3)
    ax.annotate(word, (x, y), fontsize=8, ha='center', va='bottom')
for label, color in topic_colors.items():
    ax.scatter([], [], color=color, label=label, s=80)
ax.legend(fontsize=8)
ax.axhline(0, color='lightgray', lw=0.8)
ax.axvline(0, color='lightgray', lw=0.8)
ax.set_xlabel("PCA dimension 1")
ax.set_ylabel("PCA dimension 2")
ax.set_title("Word2Vec (50d → PCA 2d)")

# --- LSA word plot (repeated for side-by-side) ---
ax = axes[1]
for word, topic in zip(target_words, ["Space"]*6+["Sports"]*6+["Cooking"]*6):
    if word in vocab:
        idx = np.where(vocab == word)[0][0]
        x, y = X_words[idx]
        ax.scatter(x, y, color=topic_colors[topic], s=60, alpha=0.8, zorder=3)
        ax.annotate(word, (x, y), fontsize=8, ha='center', va='bottom')
for label, color in topic_colors.items():
    ax.scatter([], [], color=color, label=label, s=80)
ax.legend(fontsize=8)
ax.axhline(0, color='lightgray', lw=0.8)
ax.axvline(0, color='lightgray', lw=0.8)
ax.set_xlabel("LSA dimension 1")
ax.set_ylabel("LSA dimension 2")
ax.set_title("LSA (k=2)")

plt.suptitle("Word representations - Word2Vec vs LSA on toy corpus", fontsize=12)
plt.tight_layout()
plt.show()
```



Exercise 2.1 — The Word2Vec clusters look less clean than LSA on this small corpus. Why? What would happen with 10,000 sentences instead of 18?

Exercise 2.2 — Change `sg=0` (CBOW) or `window=1` vs `window=5`. What changes in the clustering? How does window size relate to the type of similarity captured?

4.3 Part 3 — Comparison: LSA vs Word2Vec

4.3.1 3.1 — Side-by-side similarity table

```
def w2v_sim(w1, w2):
    if w1 in w2v.wv and w2 in w2v.wv:
        return float(w2v.wv.similarity(w1, w2))
    return None

all_pairs = [
    # Same-topic
    ("sun", "moon"),
    ("ball", "team"),
    ("cook", "recipe"),
    ("star", "orbit"),
    ("player", "goal"),
    ("kitchen", "ingredients"),
    # Cross-topic
    ("sun", "ball"),
    ("moon", "cook"),
    ("star", "goal"),  # 'star' is ambiguous!
    ("gravity", "recipe"),
]

print(f"{'Word pair':<30} {'LSA':>8} {'Word2Vec':>10}")
print("-" * 50)
print("--- Same-topic pairs ---")
for i, (w1, w2) in enumerate(all_pairs):
    if i == 6:
        print("--- Cross-topic pairs ---")
```

```
    ls = lsa_word_sim(w1, w2)
    ws = w2v_sim(w1, w2)
    ls_str = f"{ls:8.3f}" if ls is not None else "     N/A"
    ws_str = f"{ws:8.3f}" if ws is not None else "       N/A"
    print(f"  {w1}  {w2:<22}{ls_str}{ws_str}")
```

```
Word pair                          LSA   Word2Vec
--------------------------------------------------
--- Same-topic pairs ---
  sun   moon                     0.800   0.946
  ball   team                     0.383   0.937
  cook   recipe                     N/A       N/A
  star   orbit                   -0.213   0.959
  player   goal                      N/A       N/A
  kitchen   ingredients             1.000   0.948
--- Cross-topic pairs ---
  sun   ball                    -0.813   0.941
  moon   cook                       N/A       N/A
  star   goal                       N/A       N/A
  gravity   recipe                     N/A       N/A
```

4.3.2 3.2 — Pre-trained GloVe embeddings

Training on 18 sentences is insufficient for Word2Vec to learn meaningful representations. In practice, Word2Vec and GloVe are trained on **billions of tokens** (Wikipedia, Common Crawl).

Let's load pre-trained **GloVe** vectors (50d, trained on Wikipedia + Gigaword, ~400k vocabulary):

```
# Downloads ~66 MB on first run - cached afterwards
print("Loading pre-trained GloVe vectors (first run: ~66 MB download)...")
glove = gensim.downloader.load('glove-wiki-gigaword-50')
print(f"Loaded: {len(glove):,} words - {glove.vector_size}d vectors")
```

```
Loading pre-trained GloVe vectors (first run: ~66 MB download)...
Loaded: 400,000 words - 50d vectors
```

```
# Classic analogy: king - man + woman  queen
def analogy(model, pos1, neg1, pos2, topn=4):
    result = model.most_similar(positive=[pos1, pos2], negative=[neg1], topn=topn)
    print(f"  {pos1} - {neg1} + {pos2}  →  {[w for w, _ in result]}")

print("Analogy tests with pre-trained GloVe (A - B + C  ?)")
print()
analogy(glove, "king",   "man",     "woman")     # → queen
analogy(glove, "paris",  "france",  "italy")     # → rome
analogy(glove, "walk",   "walking", "swimming")  # → swim
analogy(glove, "better", "good",    "bad")       # → worse
analogy(glove, "doctor", "man",     "woman")     # gender bias
```

```
Analogy tests with pre-trained GloVe (A - B + C  ?)

  king - man + woman  →  ['queen', 'throne', 'prince', 'daughter']
  paris - france + italy  →  ['rome', 'milan', 'turin', 'venice']
```

```
  walk - walking + swimming  →  ['pool', 'swim', 'indoor', 'rowing']
  better - good + bad  →  ['worse', 'too', 'unfortunately', 'because']
  doctor - man + woman  →  ['nurse', 'child', 'pregnant', 'mother']
```

```python
# Three-way similarity comparison: LSA (toy) vs Word2Vec (toy) vs GloVe (large)
comparison_pairs = [
    ("sun",     "moon"),
    ("sun",     "ball"),
    ("star",    "planet"),
    ("star",    "goal"),
    ("cook",    "recipe"),
    ("cook",    "orbit"),
    ("player",  "athlete"),
    ("bread",   "flour"),
]

print(f"{'Word pair':<25} {'LSA (toy)':>11} {'W2V (toy)':>11} {'GloVe (large)':>14}")
print("-" * 63)
for w1, w2 in comparison_pairs:
    ls = lsa_word_sim(w1, w2)
    ws = w2v_sim(w1, w2)
    try:
        gs = glove.similarity(w1, w2)
    except KeyError:
        gs = None
    ls_str = f"{ls:9.3f}" if ls is not None else "        N/A"
    ws_str = f"{ws:9.3f}" if ws is not None else "        N/A"
    gs_str = f"{gs:9.3f}" if gs is not None else "           N/A"
    print(f"  {w1}   {w2:<18}{ls_str}{ws_str}{gs_str}")
```

```
Word pair                  LSA (toy)   W2V (toy)  GloVe (large)
---------------------------------------------------------------
  sun   moon                  0.800    0.946    0.654
  sun   ball                 -0.813    0.941    0.347
  star   planet                    N/A        N/A    0.476
  star   goal                      N/A        N/A    0.439
  cook   recipe                    N/A        N/A    0.620
  cook   orbit                     N/A        N/A    0.109
  player   athlete                   N/A        N/A    0.676
  bread   flour                 1.000    0.995    0.818
```

Exercise 3.1 — For which pairs do LSA and GloVe agree? For which do they disagree most? What explains the disagreements?

Exercise 3.2 — Try analogy arithmetic with your toy LSA vectors (use `X_words` and `vocab`). Does it work? Why or why not?

Exercise 3.3 — The last analogy (`doctor - man + woman`) often returns a gender-biased result. What does this tell us about what Word2Vec / GloVe capture?

4.4 Summary: LSA vs Word2Vec

Property	LSA	Word2Vec
Method	SVD on count matrix	Skip-gram neural classifier
Training	Closed-form (fast)	Gradient descent (slower)
Data needed	Works on small corpora	Needs millions of tokens
Polysemy	One vector per word (poor)	One vector per word (poor)
Analogy structure	Weak	Strong at scale
Interpretability	Moderate (SVD components)	Low
Scales with corpus	Limited by matrix size	Scales with SGD

Both methods produce a single static vector per word regardless of context. This is a fundamental limitation — the word *bank* has one vector whether it refers to a riverbank or a financial institution. **Contextualised embeddings** (ELMo, BERT, GPT) solve this by making representations context-dependent. This is exactly what the Transformer architecture achieves — next lecture.

> **Final exercise** — In what scenario would you choose LSA over Word2Vec?
> Think about: corpus size, compute budget, interpretability requirements, and domain-specific vocabulary.

Chapter 5

Transformers

5.1 Outline

- How transformers are defined mathematically and what training objectives they optimise
- How **self-attention** produces contextual representations from raw embeddings
- The role of positional encoding, FFN, skip connections, layer normalisation, and dropout
- The difference between **encoder-only**, **decoder-only**, and **encoder-decoder** architectures
- How modern LLMs are fine-tuned (SFT, RLHF) and how text is generated at inference time

---

5.2 From Turing to ChatGPT

- Deep learning architecture based on **attention mechanisms**
- Weight the importance of different tokens in a sequence to **model long-range dependencies efficiently**

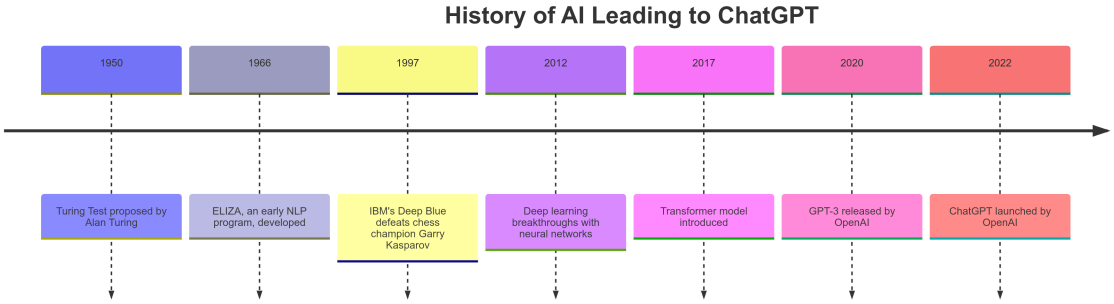
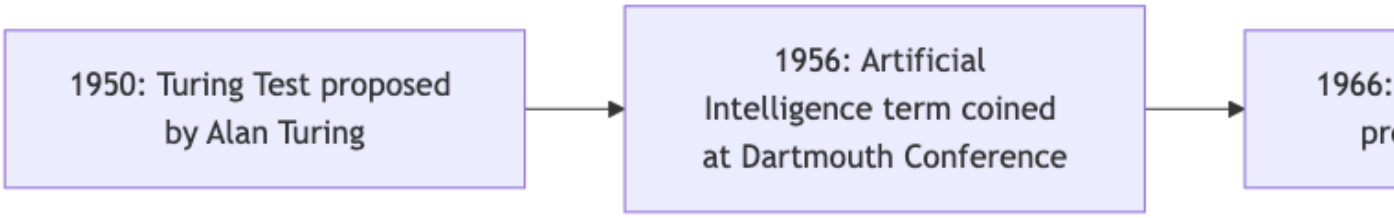


Figure 5.1: History of AI leading to chatGPT



5.3 What is a Transformer?

A classical transformer is a function

$$T(w_{1:n}) = \begin{bmatrix} p_1 \\ \vdots \\ p_{|V|} \end{bmatrix}$$

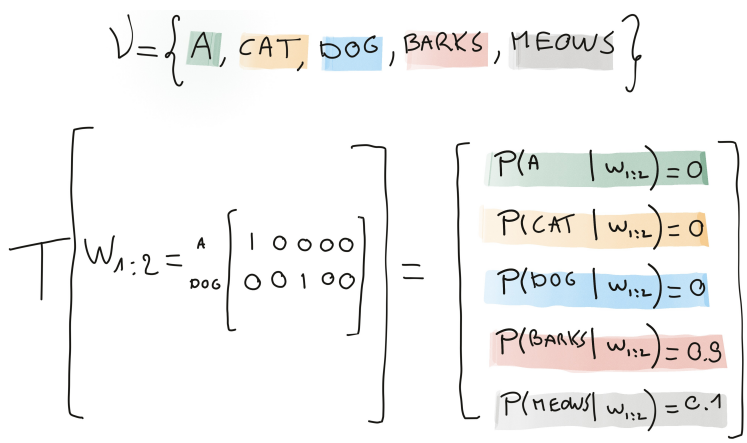


Figure 5.2: The nature of transformer

- $\mathcal{V}$ is a finite set (vocabulary) (Open AI uses 50,000 token as vocabulary)
- $w_{1:n}$ be a sequence of token, where each $w_i \in \mathcal{V}$ (In practice context varies from 1 to n, the blocksize)
- p_j is the probability of $w_{(j)}$ the jth token in $\mathcal{V}$

5.3.1 Remarks

- $w_{1:n}$ is usually coded a matrix of one-hot vectors with $w_{1:n} \in \mathbb{R}^{n \times |V|}$.
- For classical chatbot $p_j = P(w_{n+1} = w_{(j)} | w_{1:n})$ where $w_{(j)}$ is the jthe element of $\mathcal{V}$.

5.4 Optimized criteria

Transformers typically optimize different criteria depending on the task they are trained on.

5.4.1 Causal Language Modeling (CLM) / Autoregressive Loss

- **Used in:** GPT models

- **Objective:** Predict the next token given the previous tokens

The training set is a set of sequences (s_j) of n tokens $s_j = (w_1^j, w_2^j, \dots, w_n^j)$:

$$L_{CLM} = -\sum_j \sum_{t=1}^{n} \sum_{c \in V} \mathbb{I}_{w_t^j = c} \log P(w_t^j = c | w_{1:t-1}^j)$$

where $w_{1:t-1}^j$ denotes the tokens preceding position t in the sequence j

The criterion is a kind of pseudo log-likelihood.

If nothing is learned each token has the same probability of appearing: the largest possible value of the criterion is

$$-\log \frac{1}{|V|}$$

5.5 Masked Language Modeling (MLM)

- **Used in:** BERT models
- **Objective:** Predict randomly masked tokens in a sentence.

Let $M_j \subset \{1, \dots, n\}$ be the set of masked positions in the sequence (s_j):

$$L_{MLM} = -\sum_j \sum_{t \in M_j} \sum_{c \in V} \mathbb{I}_{w_t^j = c} \log P(w_t^j = c | w_{(1:n) \setminus M}^j)$$

5.6 Conditional Language Modeling / Translation Loss

- **Used in:** T5, BART
- **Objective:** Predict an output sequence given an input sequence (e.g., translation, summarization).

Given input sequence $w^j = (w_1^j, \dots, w_n^j)$ and output sequence $v^j = (v_1^j, \dots, v_m^j)$:

$$L_{Seq2Seq} = -\sum_j \sum_{t=1}^{m} \sum_{c \in V} \mathbb{I}_{v_t^j = c} \log P(v_t^j = c | v_{1:t-1}^j, w_{1:n}^j)$$

5.6.1 Other losses

Transformers exact objective functions depends on type of considered problems

5.7 Initial embedding

- The token are first embedded in a d dimensional space (see previous topic)
- the representation $x_i = Ew_i$ is independent of context.
- Let $X = x_{1:n} = w_{1:n} E^\top \in \mathbb{R}^{n \times d}$.

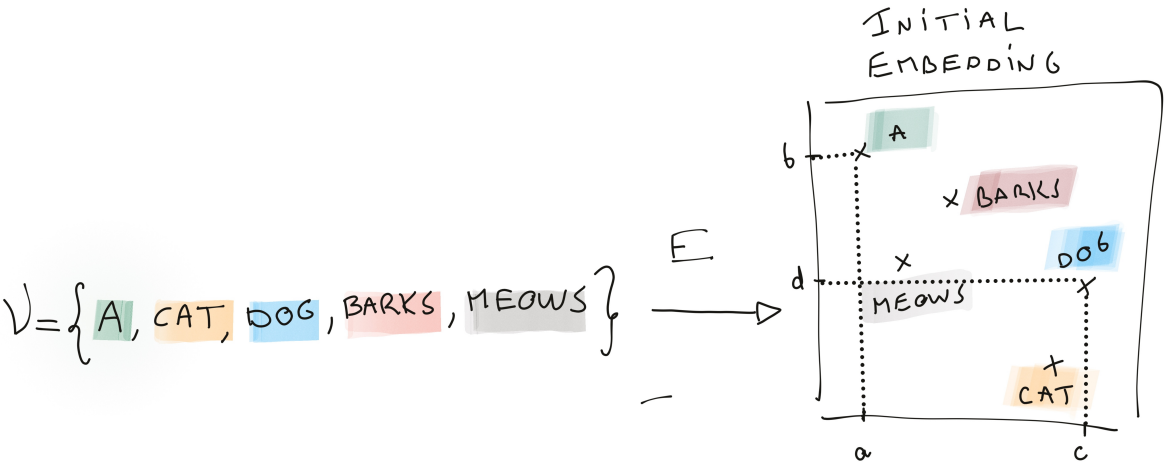


Figure 5.3: Initial embedding

5.8 A minimal self-attention architecture for decoder

Attention is a communication mecanism between the tokens of a sequence.

Self attention proposes a contextual representation h_i of x_i as a linear combination the sequence:

$$h_i = V.\sum_{j=1}^{n} \alpha_{ij} x_j$$

- h_i aggregates the past between query (current postion i) and key (other past position j)
  - $\sum_j \alpha_{ij} = 1$
- In decoder the future cannot communicate with the past
  - $\forall j > i, \alpha_{ij} = 0$
- the structure of the attention matrix is thus lower triangular in classical GPT decoder,
- it is not necessary to have a triangular attention matrix (i.e. sentiment analysis)
- V is learned matrix of parameters and can be seen a projection matrix modifying the embedding of the linear combination of sequence tokens. Let consider V as a square matrix $d \times d$.

5.9 Attention and matrix notation

- $\alpha_{ij} = \text{softmax}(x_i^T Q^T K x_j) = \frac{exp(x_i^T Q^T K x_j)}{\sum_{j=1}^{n} \exp(x_i^T Q^T K x_j)}$, where
  - Q is a matrix that modifies the embedding of the token we are looking for. Let consider Q as a square matrix $d \times d$.
    - K is a matrix that modifies the embedding of the word we are comparing against. Let consider K as a square matrix $d \times d$.

5.9.1 Matrix notation

$$H = \begin{bmatrix} h_1^T \\ \vdots \\ h_n^T \end{bmatrix} = \text{Attention(Q,K,V,X)} = \text{softmax}\left(\frac{XQ^TKX^T}{\sqrt{d_k}}\right) XV^T$$

5.9.2 Remark

In the original paper ("Attention is all you need", 2017) the notation is different and notes:

- XQ^T as Q,
- XK^T as K, and
- XV^T as V

The $1/\sqrt{d_k}$ scaling prevents the dot products from growing large in magnitude (which would push softmax into regions with very small gradients).

5.10 Illustration of self-Attention

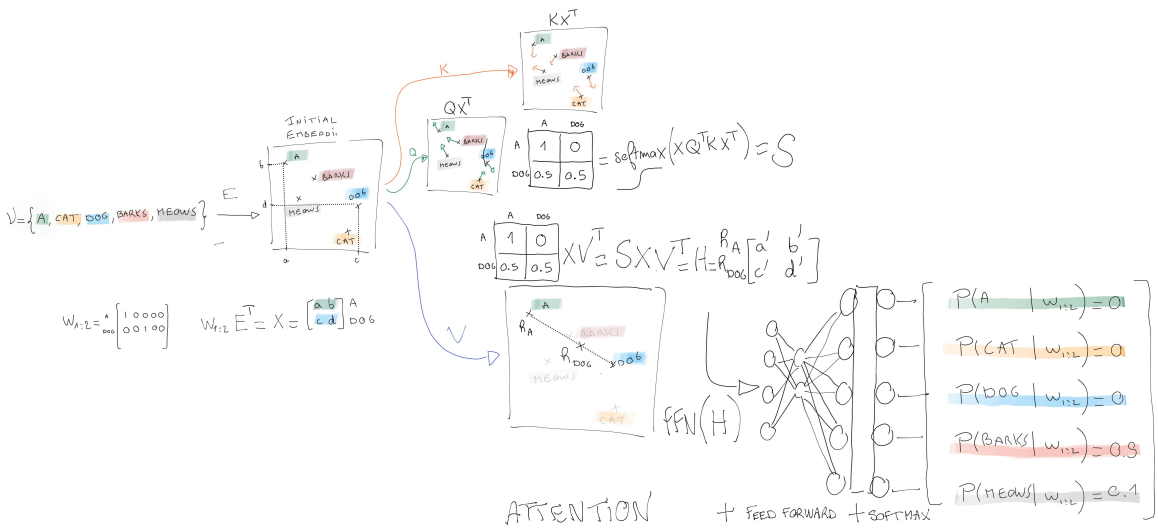


Figure 5.4: Self-attention block in auto-encoder

5.11 Interpretation of Q, K, V

- The query is modified version of an initial embedding $q_i = Qx_i$

- The Key is modified version of an initial embedding $k_i = Kx_i$
- Value is modified version of an initial embedding $v_i = Vx_i$, which should include contextual information since it is a transformation of a linear combination of the context tokens…

Attention Concept	Sometimes Similar to
Query	Target (What to focus on)
Key	Context (What to compare against, the source of information)
Value	Contextual embedding

5.12 Important details

The original paper introduces many add-ons, which much improve the transformers performances:

- **Positional encoding** for introducing the notion of order in the sequence
- FFN: **Feed Forward Neural layer**.
  - As the attention is a linear operation composed with a softmax, a Feed Forward Neural layer is added for making the function more flexible
- **Multi-blocks**: In the spirit of Deep learning blocks of attention and FNN are repeated…
- **Dropout layers** are used for preventing overfitting (from "Dropout a simple way to prevent NN from overfitting" Hinton)
- Simple trick of **Skip connection** for stabilizing the gradient (from "Deep learning residuals for image recognition")
  - The deep learning structure tends to cause problems with gradient computation (vanishing or exploding gradient)
- In order to make tokens comparable, they are all **normalized** at the output of each blocks:
  - The classical z-transformed is applied (with Bayesian correction)
- **Multi-head attention**: computes multiple attention functions (heads) in parallel

5.13 Layer Normalization

After each sub-layer (attention or FFN), a **Layer Normalization** is applied:

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sigma + \epsilon} + \beta$$

where μ and σ are the mean and standard deviation computed **over the feature dimension** (not the batch), and γ, β are learned scale and shift parameters.

Why not Batch Norm? In NLP, sequences have variable lengths and mini-batches are small; statistics computed over the batch are unreliable. LayerNorm computes statistics per token, making it robust to sequence length.

5.13.1 Place in the architecture

In the original paper the order is: sub-layer → Add (skip) → Norm. Most modern implementations use *Pre-LN*: Norm → sub-layer → Add, which stabilizes training of very deep networks.

5.14 Dropout in Transformers

Dropout (Hinton et al., 2014) randomly zeroes activations during training:

$$\text{Dropout}(x)_i = \begin{cases} 0 & \text{with probability } p \\ \frac{x_i}{1-p} & \text{otherwise} \end{cases}$$

In transformers, dropout is applied:

1. **To attention weights** A before computing the weighted sum of values.
2. **To the output of each sub-layer** before the residual addition.

Typical rates: $p \in \{0.1, 0.2\}$. Dropout is **disabled at inference time**.

5.15 Positional encoding

- Attention do not consider order of tokens !
- Incorporating information about the order of tokens is achieved by adding positional encodings to the input embeddings

The original transformer implementation uses:

- even indices:
$$PE_{\text{pos},2i} = \sin\left(\frac{\text{pos}}{10000^{\frac{2i}{d}}}\right)$$
- odd indices:
$$PE_{\text{pos},2i+1} = \cos\left(\frac{\text{pos}}{10000^{\frac{2i}{d}}}\right)$$

It ensures that each position is assigned a unique encoding

5.16 Feed Forward Neural Network

- **Attention mechanism operates linearly** to capture dependencies between tokens in a sequence
- Non-linearity enhance the model's expressive power

5.16.1 Structure of the FFN:

Each FFN consists of two linear transformations with a non-linear activation function in between:

1. First Linear Transformation: embedding of the input from the model's dimensionality d to a higher-dimensional space
2. Activation Function: A non-linear function is applied to introduce non-linearity
3. Second Linear Transformation: projects the output back to the original dimensionality

$$\text{FFN}(x) = W_2 \cdot \text{ReLU}(W_1 \cdot x + b_1) + b_2$$

where W_1 and W_2 are weight matrices, and b_1 and b_2 are bias vectors.

5.17 Skip connection (original paper "Deep Residual Learning for Image Recognition" 2015 by He et al.)

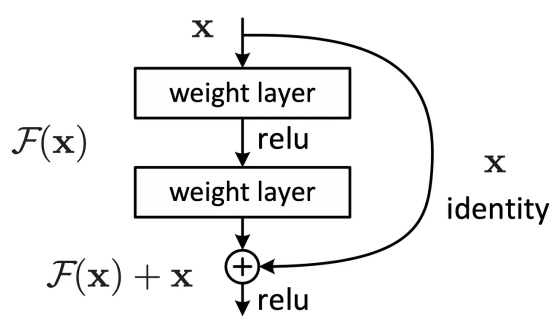


Figure 5.5: Skip connection

Skip connections address the vanishing (and exploding) gradient problems

- Consider a neural network layer with an input x and a desired underlying function $H(x)$.
- Incorporating a skip connection, the layer is restructured to model a residual $F(x) = H(x) - x$. Thus, the output of this layer becomes:

$$y = F(x) + x$$

5.17.1 Benefits:

- Enhanced Gradient Flow
- Faster convergence and higher accuracy

5.18 Multi-head attention

- Multi-Head Attention captures various aspects of the data.
- The mechanism computes multiple attention functions (heads) in parallel.
- Each head operates on linearly projected versions of the queries, keys, and values

Each head k computes the attention scores

$$\text{Attention}(Q_k, K_k, V_k) = \text{softmax}\left(\frac{XQ_k^T K_k X}{\sqrt{d_k}}\right) XV_k^T$$

where d_k is the dimension of the key vectors

5.18.1 Concatenation

The outputs of all heads are concatenated and projected through a final linear layer:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)P$$

The multi head matrix is $d \times d$, where - h is the number of head, - $d_k = d/h$ the size of the projection space of head k - P is a projection matrix combining the heads.

5.18.2 Benefits

- Diverse Representations: Each head can capture different features or relationships within the data, enabling the model to understand various aspects of the input (grammar, style, "meaning"...).
  - Parallel Processing: Multiple heads allow the model to process different parts of the sequence simultaneously, improving efficiency.
  - Enhanced Capacity: By attending to information from different subspaces, the model can capture complex patterns and dependencies.

5.19 Complexity of Self-Attention

Operation	Time complexity	Space complexity
Self-attention	$O(n^2 d)$	$O(n^2)$
FFN	$O(nd^2)$	$O(d^2)$

Self-attention is **quadratic in sequence length** n. For long documents ($n > 4096$) this becomes prohibitive — motivation for variants such as:

- **Sparse attention** (Longformer, BigBird)
- **Linear attention** approximations
- **Flash Attention** (IO-aware exact attention, Dao et al. 2022)

Modern LLMs also use a **KV-cache** at inference: keys and values from past tokens are cached so each new token only attends to stored K/V pairs, reducing per-step cost from $O(n^2)$ to $O(n)$.

5.20 Three Transformer Families

Architecture	Attention mask	Representative models	Typical use
Encoder-only	Bidirectional (full)	BERT, RoBERTa	Classification, NER, Q&A
Decoder-only	Causal (triangular)	GPT-2/3/4, LLaMA	Text generation, chat
Encoder-Decoder	Enc: full; Dec: causal + cross-attn	T5, BART, mT5	Translation, summarization

Cross-attention (encoder-decoder models): the decoder's Q comes from the decoder hidden state, while K and V come from the encoder output. This is the original architecture of "Attention is All You Need" (Vaswani et al., 2017).

5.21 Tokenization

Transformers do not operate on raw characters or words but on **subword tokens**.

5.21.1 Byte-Pair Encoding (BPE)

1. Start with a character-level vocabulary.
2. Iteratively merge the most frequent adjacent pair of tokens.
3. Repeat until the vocabulary reaches size $|V|$.

Example: `"lower"` → `["low", "er"]`; `"lowest"` → `["low", "est"]`

GPT-2/3/4 use BPE with $|V| \approx 50{,}000$. BERT uses **WordPiece** (similar but merges maximise likelihood rather than frequency).

5.21.2 Why subwords?

- Handles unknown words (OOV) via decomposition.
- Balances vocabulary size vs. sequence length.
- Works across languages without language-specific tokenisers.

5.22 Fine tuning

After a pretraining step that resembles closely to a decoder…

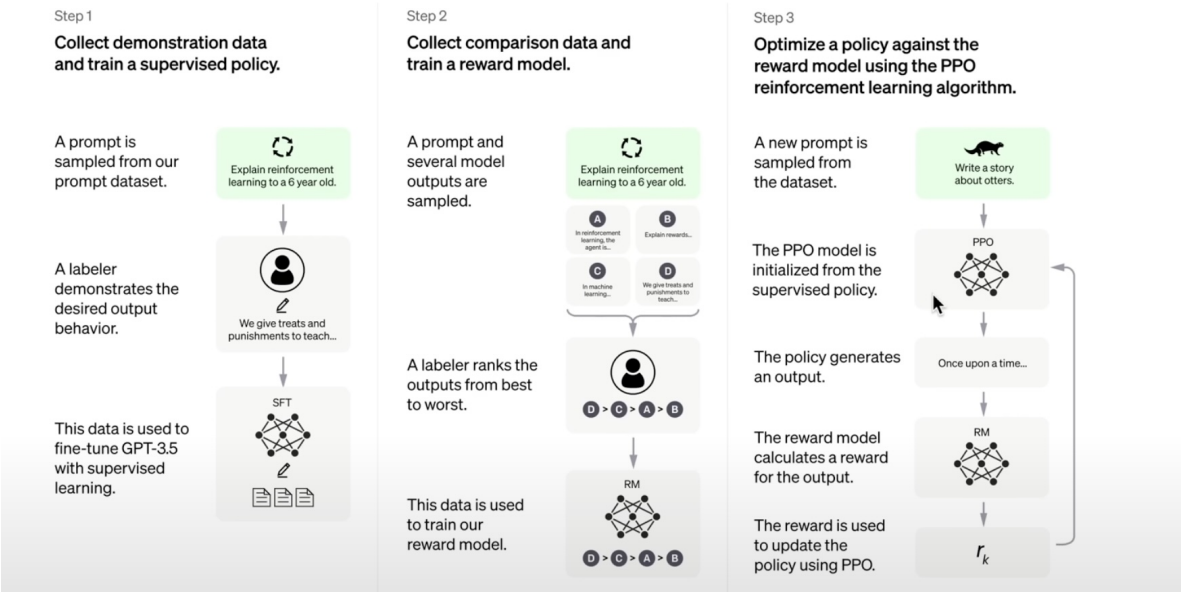


Figure 5.6: ChatGPT fine tuning (from A. Karpathy lecture)

The modern fine-tuning pipeline consists of three stages:

1. **Pre-training** — CLM on a large corpus; the model learns general language structure.
2. **Supervised Fine-Tuning (SFT)** — fine-tune on curated (prompt, response) pairs to teach the desired format and style.
3. **Reinforcement Learning from Human Feedback (RLHF)** — a reward model is trained on human preference rankings; the LLM policy is then optimised with PPO to maximise reward while staying close to the SFT model (KL penalty).

5.23 Generating Text from a Decoder

Given $P(w_{n+1} \mid w_{1:n})$, how do we select the next token?

5.23.1 Greedy decoding

Always pick $\arg\max_w P(w \mid w_{1:n})$. Fast but repetitive — the model tends to loop.

5.23.2 Beam search

Maintain B candidate sequences, expand each, keep the B highest probability continuations. Better quality but still deterministic.

5.23.3 Sampling with temperature

$$P_T(w) \propto P(w)^{1/T}$$

- $T \to 0$: greedy (deterministic)
- $T = 1$: original distribution

- $T > 1$: flatter, more random

5.23.4 Top-k and Top-p (nucleus) sampling

Restrict sampling to the k most probable tokens, or to the smallest set whose cumulative probability exceeds p. In practice, `top_p=0.9, temperature=0.8` is a common default for creative tasks.

5.24 From Theory to Code

The TP notebook `gpt_dev.ipynb` implements every concept in this chapter:

Concept	Location in notebook
Bigram baseline	Section 1
Token + positional embedding	Section 3
Self-attention head	`Head` class
Multi-head attention	`MultiHeadAttention` class
FFN + skip + LayerNorm	`Block` class
Full GPT training loop	Section 5

The implementation follows Karpathy's *nanoGPT* and trains on the Tiny Shakespeare dataset (`input.txt`).

> **Tip**
>
> Read the notebook alongside this chapter — each class maps directly to a formula above.

Chapter 6

Exercises — GPT from scratch

These exercises follow the notebook `TP/gpt_dev.ipynb` step by step, building a small GPT character-level language model from scratch in PyTorch, following Karpathy's "Let's build GPT" series.

6.1 Links

- Tiny Shakespeare dataset
- Google Colab for the video
- GitHub repo for the video
- nanoGPT repo

6.2 Supplementary links

- Attention is All You Need — Vaswani et al. (2017)
- OpenAI GPT-3 paper
- OpenAI ChatGPT blog post

6.2.1 1) Data preparation and character-level tokenizer

Notebook cells: 1 – 8

Goal: Load a raw text corpus, build a character-level vocabulary, and encode the full dataset as a sequence of integers.

Key ideas

- The tiny Shakespeare dataset contains 1,115,394 characters drawn from 65 unique characters (letters, punctuation, newlines).
- A character-level tokenizer maps each character to an integer: `encode` turns a string into a list of integers, `decode` reverses it.
- The full corpus is stored as a `torch.Tensor` of dtype `torch.long`.
- The data is split 90 % train / 10 % validation *before* any model is built.

Exercises

1. Print the vocabulary (all 65 characters). Which characters appear? Is there anything surprising?
2. Encode the string `"Hello GPT"` and decode it back. Verify the round-trip.
3. What would change if you used a word-level tokenizer instead? Estimate the new vocabulary size and the new sequence length for the same corpus.
4. The split is done at position `int(0.9 * len(data))`. Why is it important *not* to shuffle the data before splitting?

6.2.2 2) Batch loading and the context window

Notebook cells: 9 – 12

Goal: Understand how a single `(input, target)` tensor pair encodes `block_size` training examples simultaneously.

Key ideas

- `block_size` is the maximum context length (number of tokens fed to the model at once).
- For a sequence of length `block_size + 1`, there are `block_size` overlapping `(context, target)` pairs: when the input is `x[:t+1]`, the target is `x[t+1]`.
- `get_batch` samples `batch_size` random starting positions and stacks the corresponding windows into tensors `(B, T)`.

Exercises

1. Run `get_batch('train')` and print `xb`, `yb`. For the first sequence in the batch, manually list all `(context, target)` pairs.
2. Why does `get_batch` sample starting positions from `range(len(data) - block_size)` rather than `range(len(data))`?
3. Change `block_size` to 4 and `batch_size` to 2 and rerun. How does this affect the number of training examples per batch?

6.2.3 3) Bigram language model

Notebook cells: 13 – 16

Goal: Implement the simplest possible language model as a baseline before introducing attention.

Key ideas

- A bigram model predicts the next character using only the current character: the token embedding table directly provides the logits for the next token.
- The loss is cross-entropy between the predicted distribution and the true next character.
- After 100 steps of AdamW (learning rate `1e-3`, batch size 32), the loss drops from 4.88 to 4.66, and the model generates readable (but random) character sequences.
- Generation uses multinomial sampling from the softmax distribution, appending one token at a time.

Exercises

1. What is the cross-entropy loss of a *uniform* random model over 65 characters? Compare with the initial loss of 4.88.
2. The `generate` method uses the whole sequence as context, but the bigram model only ever looks at the last token. Why is this not a problem here, but would be a problem for a Transformer?

3. Train for 1000 steps instead of 100. Does the loss continue to decrease? Generate a sample and comment on its quality.
4. Modify `generate` to use greedy decoding (argmax) instead of sampling. How does the output change?

6.2.4 4) The mathematical trick in self-attention

Notebook cells: 17 – 22

Goal: Understand weighted aggregation of past tokens via matrix multiplication, the foundation of self-attention.

Key ideas

- A token at position t should only communicate with positions $\leq t$ (causal constraint).
- The simplest aggregation: $x_t^{\text{bow}} = \frac{1}{t}\sum_{i\leq t} x_i$ (bag-of-words average).
- **Version 1** (loop): explicit `for` loop over positions — correct but slow.
- **Version 2** (matrix multiply): replace the loop with a lower-triangular matrix multiply. If `wei` is the $(T \times T)$ lower-triangular matrix normalised by row, then `wei @ x` computes all position averages in one operation.
- **Version 3** (softmax): initialise `wei = zeros`, fill future positions with `-inf`, then apply softmax — this is exactly the uniform-attention baseline.

Exercises

1. Verify that versions 1, 2 and 3 produce identical outputs using `torch.allclose`.
2. Print the weight matrix `wei` for version 2 (shape $T \times T$). What does each row represent?
3. What happens if you remove the masking step in version 3? What values does softmax assign to future positions?
4. In version 2, why must the rows be normalised (divided by their sum)? What would happen to the magnitude of `xbow` without normalisation?

6.2.5 5) Self-attention with Query, Key and Value

Notebook cells: 23 – 25

Goal: Replace the uniform average with data-dependent attention weights computed from learned projections.

Key ideas

- Each position produces three vectors from the input x: a **query** $q = Qx$ (what am I looking for?), a **key** $k = Kx$ (what do I contain?), and a **value** $v = Vx$ (what do I communicate?).
- Attention scores: $\text{wei} = qk^T/\sqrt{d_k}$ — the dot product between the query at position t and the keys at all positions $\leq t$.
- After masking and softmax, the output is a weighted sum of values: $\text{out} = \text{wei} \cdot v$.
- **Self-attention**: Q, K, V are all produced from the same source x.
- **Encoder vs decoder**: removing the triangular mask gives an encoder block (all positions visible); keeping it gives a decoder block (causal, used for language modelling).

Exercises

1. Print `wei[0]` (attention weights for the first sequence in the batch). Verify that the upper triangle is exactly 0. Which position attends most to which?
2. The scaling factor is $1/\sqrt{d_k}$ (`head_size**-0.5`). Cells 26–31 show why: compute `wei.var()` with and without the scaling. What happens to the softmax distribution when the variance is large?
3. Remove the causal mask (delete the `masked_fill` line). The model can now look into the future. Why is this valid for an encoder but not for a language model?
4. Change `head_size` to 1, then to 64. How does the shape of `out` change?

6.2.6 6) Scaled attention and numerical stability

Notebook cells: 26 – 31

Goal: Understand why the $1/\sqrt{d_k}$ scaling prevents the softmax from saturating.

Key ideas

- If q and k are unit-variance, then $q \cdot k$ has variance d_k — its standard deviation grows with dimension.
- Without scaling, the dot products become large in magnitude, softmax converges to a one-hot distribution ("peaky"), and gradients vanish.
- With scaling by $1/\sqrt{d_k}$, `wei` stays unit-variance regardless of `head_size`.

Exercises

1. Run `k.var()`, `q.var()`, `wei.var()` before and after applying `* head_size**-0.5`. Confirm that scaling restores unit variance.
2. Compare `torch.softmax(tensor * 1, dim=-1)` and `torch.softmax(tensor * 8, dim=-1)` for the vector `[0.1, -0.2, 0.3, -0.2, 0.5]`. Describe the difference.
3. At initialisation, why do we want the attention weights to be approximately uniform (not peaky)?

6.2.7 7) Layer normalisation

Notebook cells: 32 – 34

Goal: Implement Layer Normalisation and understand how it differs from Batch Normalisation.

Key ideas

- LayerNorm normalises each sample independently across its feature dimension (not across the batch).
- For input x of shape (B, C): $\hat{x} = (x - \mu_x)/\sigma_x$, then out $= \gamma\hat{x} + \beta$ with learned scale γ and shift β.
- After LayerNorm, `x[0,:].mean()` `0` and `x[0,:].std()` `1` for every sample.
- In the modern Transformer ("Pre-LN"), LayerNorm is applied *before* attention and feed-forward (not after as in the original paper).

Exercises

1. Run the `LayerNorm1d` example with `x = torch.randn(32, 100)`. Verify that `x[0,:].mean()` and `x[0,:].std()` are approximately 0 and 1.
2. Check `x[:,0].mean()` and `x[:,0].std()` (one feature across the batch). Why are these *not* normalised?

3. What is the difference between this and `nn.BatchNorm1d`? In which training scenarios does LayerNorm have an advantage?

6.2.8 8) Full GPT implementation

Notebook cells: 36 – 37

Goal: Assemble all components into a complete character-level GPT and train it.

Architecture

```
Token embedding  (vocab_size → n_embd)
Position embedding (block_size → n_embd)
     N × Block:
         Pre-LN → MultiHeadAttention (n_head heads of size n_embd/n_head) → residual
         Pre-LN → FeedForward (n_embd → 4·n_embd → n_embd, ReLU) → residual
Final LayerNorm
Linear projection  (n_embd → vocab_size)
```

Hyperparameters (notebook defaults)

Parameter	Value
`batch_size`	16
`block_size`	32
`n_embd`	64
`n_head`	4
`n_layer`	4
`learning_rate`	1e-3
`max_iters`	5000
`dropout`	0.0

Results: 0.21 M parameters — train loss 1.66 / val loss 1.82 after 5000 steps.

Exercises

1. Count the parameters in each component (`Head`, `MultiHeadAttention`, `FeedFoward`, `Block`). Verify the total of 0.21 M.
2. The `Block` applies LayerNorm *before* the sub-layer (`x + self.sa(self.ln1(x))`). Swap to Post-LN (`self.ln1(x + self.sa(x))`). Does training still converge?
3. Set `dropout = 0.2` and retrain. Does the gap between train loss and val loss decrease?
4. Double `n_embd` to 128 and `n_layer` to 6. How many parameters does the model have now? What train/val loss do you achieve?
5. Train on a different text corpus (a novel, your own text). How does the generated output change?

Chapter 7

NanoGPT

Companion notebook to the Zero To Hero video on GPT.

```
# We always start with a dataset to train on. Let's download the tiny shakespeare dataset
!wget https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt
```

```
--2026-03-10 14:05:29--  https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespe
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133,
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected
HTTP request sent, awaiting response... 200 OK
Length: 1115394 (1.1M) [text/plain]
Saving to: ‘input.txt’

input.txt           100%[===================>]   1.06M  --.-KB/s    in 0.05s

2026-03-10 14:05:29 (21.6 MB/s) - ‘input.txt’ saved [1115394/1115394]
```

```
# read it in to inspect it
with open('input.txt', 'r', encoding='utf-8') as f:
    text = f.read()
```

```
print("length of dataset in characters: ", len(text))
```

```
length of dataset in characters:  1115394
```

```
# let's look at the first 1000 characters
print(text[:1000])
```

```
First Citizen:
Before we proceed any further, hear me speak.

All:
Speak, speak.

First Citizen:
You are all resolved rather to die than to famish?

All:
```

```
Resolved. resolved.

First Citizen:
First, you know Caius Marcius is chief enemy to the people.

All:
We know't, we know't.

First Citizen:
Let us kill him, and we'll have corn at our own price.
Is't a verdict?

All:
No more talking on't; let it be done: away, away!

Second Citizen:
One word, good citizens.

First Citizen:
We are accounted poor citizens, the patricians good.
What authority surfeits on would relieve us: if they
would yield us but the superfluity, while it were
wholesome, we might guess they relieved us humanely;
but they think we are too dear: the leanness that
afflicts us, the object of our misery, is as an
inventory to particularise their abundance; our
sufferance is a gain to them Let us revenge this with
our pikes, ere we become rakes: for the gods know I
speak this in hunger for bread, not in thirst for revenge.
```

```python
# here are all the unique characters that occur in this text
chars = sorted(list(set(text)))
vocab_size = len(chars)
print(''.join(chars))
print(vocab_size)
```

```
 !$&',-.3:;?ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz
65
```

```python
# create a mapping from characters to integers
stoi = { ch:i for i,ch in enumerate(chars) }
itos = { i:ch for i,ch in enumerate(chars) }
encode = lambda s: [stoi[c] for c in s] # encoder: take a string, output a list of integers
decode = lambda l: ''.join([itos[i] for i in l]) # decoder: take a list of integers, output a string

print(encode("hii there"))
print(decode(encode("hii there")))
```

```
[46, 47, 47, 1, 58, 46, 43, 56, 43]
hii there
```

7.1 Comprehension Questions — Tokenization

1. **Vocabulary size**: This notebook uses a character-level vocabulary of 65 tokens. What are the trade-offs of character-level tokenization compared to word-level or subword tokenization (e.g. BPE used in GPT-2)?
2. **Encoding**: What does `encode("Hello")` return? Explain what happens step by step.
3. **Invertibility**: Is `decode(encode(s)) == s` always true for any string `s`? Why or why not?
4. **Out-of-vocabulary**: What happens if you call `encode` on a character that is not in `chars`? How would you fix this?

```
# let's now encode the entire text dataset and store it into a torch.Tensor
import torch # we use PyTorch: https://pytorch.org
data = torch.tensor(encode(text), dtype=torch.long)
print(data.shape, data.dtype)
print(data[:1000]) # the 1000 characters we looked at earier will to the GPT look like this
```

```
torch.Size([1115394]) torch.int64
tensor([18, 47, 56, 57, 58,  1, 15, 47, 58, 47, 64, 43, 52, 10,  0, 14, 43, 44,
        53, 56, 43,  1, 61, 43,  1, 54, 56, 53, 41, 43, 43, 42,  1, 39, 52, 63,
         1, 44, 59, 56, 58, 46, 43, 56,  6,  1, 46, 43, 39, 56,  1, 51, 43,  1,
        57, 54, 43, 39, 49,  8,  0,  0, 13, 50, 50, 10,  0, 31, 54, 43, 39, 49,
         6,  1, 57, 54, 43, 39, 49,  8,  0,  0, 18, 47, 56, 57, 58,  1, 15, 47,
        58, 47, 64, 43, 52, 10,  0, 37, 53, 59,  1, 39, 56, 43,  1, 39, 50, 50,
         1, 56, 43, 57, 53, 50, 60, 43, 42,  1, 56, 39, 58, 46, 43, 56,  1, 58,
        53,  1, 42, 47, 43,  1, 58, 46, 39, 52,  1, 58, 53,  1, 44, 39, 51, 47,
        57, 46, 12,  0,  0, 13, 50, 50, 10,  0, 30, 43, 57, 53, 50, 60, 43, 42,
         8,  1, 56, 43, 57, 53, 50, 60, 43, 42,  8,  0,  0, 18, 47, 56, 57, 58,
         1, 15, 47, 58, 47, 64, 43, 52, 10,  0, 18, 47, 56, 57, 58,  6,  1, 63,
        53, 59,  1, 49, 52, 53, 61,  1, 15, 39, 47, 59, 57,  1, 25, 39, 56, 41,
        47, 59, 57,  1, 47, 57,  1, 41, 46, 47, 43, 44,  1, 43, 52, 43, 51, 63,
         1, 58, 53,  1, 58, 46, 43,  1, 54, 43, 53, 54, 50, 43,  8,  0,  0, 13,
        50, 50, 10,  0, 35, 43,  1, 49, 52, 53, 61,  5, 58,  6,  1, 61, 43,  1,
        49, 52, 53, 61,  5, 58,  8,  0,  0, 18, 47, 56, 57, 58,  1, 15, 47, 58,
        47, 64, 43, 52, 10,  0, 24, 43, 58,  1, 59, 57,  1, 49, 47, 50, 50,  1,
        46, 47, 51,  6,  1, 39, 52, 42,  1, 61, 43,  5, 50, 50,  1, 46, 39, 60,
        43,  1, 41, 53, 56, 52,  1, 39, 58,  1, 53, 59, 56,  1, 53, 61, 52,  1,
        54, 56, 47, 41, 43,  8,  0, 21, 57,  5, 58,  1, 39,  1, 60, 43, 56, 42,
        47, 41, 58, 12,  0,  0, 13, 50, 50, 10,  0, 26, 53,  1, 51, 53, 56, 43,
         1, 58, 39, 50, 49, 47, 52, 45,  1, 53, 52,  5, 58, 11,  1, 50, 43, 58,
         1, 47, 58,  1, 40, 43,  1, 42, 53, 52, 43, 10,  1, 39, 61, 39, 63,  6,
         1, 39, 61, 39, 63,  2,  0,  0, 31, 43, 41, 53, 52, 42,  1, 15, 47, 58,
        47, 64, 43, 52, 10,  0, 27, 52, 43,  1, 61, 53, 56, 42,  6,  1, 45, 53,
        53, 42,  1, 41, 47, 58, 47, 64, 43, 52, 57,  8,  0,  0, 18, 47, 56, 57,
        58,  1, 15, 47, 58, 47, 64, 43, 52, 10,  0, 35, 43,  1, 39, 56, 43,  1,
        39, 41, 41, 53, 59, 52, 58, 43, 42,  1, 54, 53, 53, 56,  1, 41, 47, 58,
        47, 64, 43, 52, 57,  6,  1, 58, 46, 43,  1, 54, 39, 58, 56, 47, 41, 47,
        39, 52, 57,  1, 45, 53, 53, 42,  8,  0, 35, 46, 39, 58,  1, 39, 59, 58,
        46, 53, 56, 47, 58, 63,  1, 57, 59, 56, 44, 43, 47, 58, 57,  1, 53, 52,
         1, 61, 53, 59, 50, 42,  1, 56, 43, 50, 47, 43, 60, 43,  1, 59, 57, 10,
         1, 47, 44,  1, 58, 46, 43, 63,  0, 61, 53, 59, 50, 42,  1, 63, 47, 43,
```

```
        50, 42,  1, 59, 57,  1, 40, 59, 58,  1, 58, 46, 43,  1, 57, 59, 54, 43,
        56, 44, 50, 59, 47, 58, 63,  6,  1, 61, 46, 47, 50, 43,  1, 47, 58,  1,
        61, 43, 56, 43,  0, 61, 46, 53, 50, 43, 57, 53, 51, 43,  6,  1, 61, 43,
         1, 51, 47, 45, 46, 58,  1, 45, 59, 43, 57, 57,  1, 58, 46, 43, 63,  1,
        56, 43, 50, 47, 43, 60, 43, 42,  1, 59, 57,  1, 46, 59, 51, 39, 52, 43,
        50, 63, 11,  0, 40, 59, 58,  1, 58, 46, 43, 63,  1, 58, 46, 47, 52, 49,
         1, 61, 43,  1, 39, 56, 43,  1, 58, 53, 53,  1, 42, 43, 39, 56, 10,  1,
        58, 46, 43,  1, 50, 43, 39, 52, 52, 43, 57, 57,  1, 58, 46, 39, 58,  0,
        39, 44, 44, 50, 47, 41, 58, 57,  1, 59, 57,  6,  1, 58, 46, 43,  1, 53,
        40, 48, 43, 41, 58,  1, 53, 44,  1, 53, 59, 56,  1, 51, 47, 57, 43, 56,
        63,  6,  1, 47, 57,  1, 39, 57,  1, 39, 52,  0, 47, 52, 60, 43, 52, 58,
        53, 56, 63,  1, 58, 53,  1, 54, 39, 56, 58, 47, 41, 59, 50, 39, 56, 47,
        57, 43,  1, 58, 46, 43, 47, 56,  1, 39, 40, 59, 52, 42, 39, 52, 41, 43,
        11,  1, 53, 59, 56,  0, 57, 59, 44, 44, 43, 56, 39, 52, 41, 43,  1, 47,
        57,  1, 39,  1, 45, 39, 47, 52,  1, 58, 53,  1, 58, 46, 43, 51,  1, 24,
        43, 58,  1, 59, 57,  1, 56, 43, 60, 43, 52, 45, 43,  1, 58, 46, 47, 57,
         1, 61, 47, 58, 46,  0, 53, 59, 56,  1, 54, 47, 49, 43, 57,  6,  1, 43,
        56, 43,  1, 61, 43,  1, 40, 43, 41, 53, 51, 43,  1, 56, 39, 49, 43, 57,
        10,  1, 44, 53, 56,  1, 58, 46, 43,  1, 45, 53, 42, 57,  1, 49, 52, 53,
        61,  1, 21,  0, 57, 54, 43, 39, 49,  1, 58, 46, 47, 57,  1, 47, 52,  1,
        46, 59, 52, 45, 43, 56,  1, 44, 53, 56,  1, 40, 56, 43, 39, 42,  6,  1,
        52, 53, 58,  1, 47, 52,  1, 58, 46, 47, 56, 57, 58,  1, 44, 53, 56,  1,
        56, 43, 60, 43, 52, 45, 43,  8,  0,  0])
```

```
# Let's now split up the data into train and validation sets
n = int(0.9*len(data)) # first 90% will be train, rest val
train_data = data[:n]
val_data = data[n:]
```

```
block_size = 8
train_data[:block_size+1]
```

```
tensor([18, 47, 56, 57, 58,  1, 15, 47, 58])
```

7.2 Comprehension Questions — Training Objective

1. **Context–target pairs**: A single sequence of length `block_size + 1` yields `block_size` training examples. Why is training on all these sub-contexts simultaneously beneficial compared to only using the full context?

2. **Causal structure**: Why must the model never see tokens *after* position `t` when predicting position `t`? What mechanism will enforce this later in the Transformer?

3. **`block_size`**: What happens if you increase `block_size` from 8 to 256? What changes in terms of memory, computation, and the type of dependencies the model can learn?

```
x = train_data[:block_size]
y = train_data[1:block_size+1]
for t in range(block_size):
    context = x[:t+1]
    target = y[t]
    print(f"when input is {context} the target: {target}")
```

```
when input is tensor([18]) the target: 47
when input is tensor([18, 47]) the target: 56
when input is tensor([18, 47, 56]) the target: 57
when input is tensor([18, 47, 56, 57]) the target: 58
when input is tensor([18, 47, 56, 57, 58]) the target: 1
when input is tensor([18, 47, 56, 57, 58,  1]) the target: 15
when input is tensor([18, 47, 56, 57, 58,  1, 15]) the target: 47
when input is tensor([18, 47, 56, 57, 58,  1, 15, 47]) the target: 58
```

```python
torch.manual_seed(1337)
batch_size = 4 # how many independent sequences will we process in parallel?
block_size = 8 # what is the maximum context length for predictions?

def get_batch(split):
    # generate a small batch of data of inputs x and targets y
    data = train_data if split == 'train' else val_data
    ix = torch.randint(len(data) - block_size, (batch_size,))
    x = torch.stack([data[i:i+block_size] for i in ix])
    y = torch.stack([data[i+1:i+block_size+1] for i in ix])
    return x, y

xb, yb = get_batch('train')
print('inputs:')
print(xb.shape)
print(xb)
print('targets:')
print(yb.shape)
print(yb)

print('----')

for b in range(batch_size): # batch dimension
    for t in range(block_size): # time dimension
        context = xb[b, :t+1]
        target = yb[b,t]
        print(f"when input is {context.tolist()} the target: {target}")
```

```
inputs:
torch.Size([4, 8])
tensor([[24, 43, 58,  5, 57,  1, 46, 43],
        [44, 53, 56,  1, 58, 46, 39, 58],
        [52, 58,  1, 58, 46, 39, 58,  1],
        [25, 17, 27, 10,  0, 21,  1, 54]])
targets:
torch.Size([4, 8])
tensor([[43, 58,  5, 57,  1, 46, 43, 39],
        [53, 56,  1, 58, 46, 39, 58,  1],
        [58,  1, 58, 46, 39, 58,  1, 46],
        [17, 27, 10,  0, 21,  1, 54, 39]])
----
when input is [24] the target: 43
```

```
when input is [24, 43] the target: 58
when input is [24, 43, 58] the target: 5
when input is [24, 43, 58, 5] the target: 57
when input is [24, 43, 58, 5, 57] the target: 1
when input is [24, 43, 58, 5, 57, 1] the target: 46
when input is [24, 43, 58, 5, 57, 1, 46] the target: 43
when input is [24, 43, 58, 5, 57, 1, 46, 43] the target: 39
when input is [44] the target: 53
when input is [44, 53] the target: 56
when input is [44, 53, 56] the target: 1
when input is [44, 53, 56, 1] the target: 58
when input is [44, 53, 56, 1, 58] the target: 46
when input is [44, 53, 56, 1, 58, 46] the target: 39
when input is [44, 53, 56, 1, 58, 46, 39] the target: 58
when input is [44, 53, 56, 1, 58, 46, 39, 58] the target: 1
when input is [52] the target: 58
when input is [52, 58] the target: 1
when input is [52, 58, 1] the target: 58
when input is [52, 58, 1, 58] the target: 46
when input is [52, 58, 1, 58, 46] the target: 39
when input is [52, 58, 1, 58, 46, 39] the target: 58
when input is [52, 58, 1, 58, 46, 39, 58] the target: 1
when input is [52, 58, 1, 58, 46, 39, 58, 1] the target: 46
when input is [25] the target: 17
when input is [25, 17] the target: 27
when input is [25, 17, 27] the target: 10
when input is [25, 17, 27, 10] the target: 0
when input is [25, 17, 27, 10, 0] the target: 21
when input is [25, 17, 27, 10, 0, 21] the target: 1
when input is [25, 17, 27, 10, 0, 21, 1] the target: 54
when input is [25, 17, 27, 10, 0, 21, 1, 54] the target: 39
```

7.3 Comprehension Questions — Bigram Language Model

1. **Model capacity**: The bigram model uses a single `nn.Embedding(vocab_size, vocab_size)` table. How many parameters does it have? What is the maximum amount of information it can capture about the language?

2. **Initial loss**: With `vocab_size = 65`, what is the theoretical loss of a randomly initialized model (before training)? Hint: think in terms of the uniform distribution over 65 classes. How does this compare to the observed value of ~4.87?

3. **`generate` function**: Trace through one step of `generate`. Starting from `idx` of shape `(1, 1)`, what are the shapes of `logits`, `probs`, `idx_next`, and the new `idx` after one iteration?

4. **Limitation**: The bigram model ignores all context except the previous token. What structural change would be needed to condition on the last `k` tokens instead?

```
print(xb) # our input to the transformer
```

```
tensor([[24, 43, 58,  5, 57,  1, 46, 43],
        [44, 53, 56,  1, 58, 46, 39, 58],
```

```
        [52, 58,  1, 58, 46, 39, 58,  1],
        [25, 17, 27, 10,  0, 21,  1, 54]])
```

```python
import torch
import torch.nn as nn
from torch.nn import functional as F
torch.manual_seed(1337)

class BigramLanguageModel(nn.Module):

    def __init__(self, vocab_size):
        super().__init__()
        # each token directly reads off the logits for the next token from a lookup table
        self.token_embedding_table = nn.Embedding(vocab_size, vocab_size)

    def forward(self, idx, targets=None):

        # idx and targets are both (B,T) tensor of integers
        logits = self.token_embedding_table(idx) # (B,T,C)

        if targets is None:
            loss = None
        else:
            B, T, C = logits.shape
            logits = logits.view(B*T, C)
            targets = targets.view(B*T)
            loss = F.cross_entropy(logits, targets)

        return logits, loss

    def generate(self, idx, max_new_tokens):
        # idx is (B, T) array of indices in the current context
        for _ in range(max_new_tokens):
            # get the predictions
            logits, loss = self(idx)
            # focus only on the last time step
            logits = logits[:, -1, :] # becomes (B, C)
            # apply softmax to get probabilities
            probs = F.softmax(logits, dim=-1) # (B, C)
            # sample from the distribution
            idx_next = torch.multinomial(probs, num_samples=1) # (B, 1)
            # append sampled index to the running sequence
            idx = torch.cat((idx, idx_next), dim=1) # (B, T+1)
        return idx

m = BigramLanguageModel(vocab_size)
logits, loss = m(xb, yb)
print(logits.shape)
print(loss)

print(decode(m.generate(idx = torch.zeros((1, 1), dtype=torch.long), max_new_tokens=100)[0].tolist()))
```

```
torch.Size([32, 65])
tensor(4.8786, grad_fn=<NllLossBackward0>)

SKIcLT;AcELMoTbvZv C?nq-QE33:CJqkOKH-q;:la!oiywkHjgChzbQ?u!3bLIgwevmyFJGUGp
wnYWmnxKWWev-tDqXErVKLgJ
```

```
# create a PyTorch optimizer
optimizer = torch.optim.AdamW(m.parameters(), lr=1e-3)
```

```
batch_size = 32
for steps in range(100): # increase number of steps for good results...

    # sample a batch of data
    xb, yb = get_batch('train')

    # evaluate the loss
    logits, loss = m(xb, yb)
    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    optimizer.step()

print(loss.item())
```

```
4.65630578994751
```

```
print(decode(m.generate(idx = torch.zeros((1, 1), dtype=torch.long), max_new_tokens=500)[0].tolist())
```

```
oTo.JUZ!!zqe!
xBP qbs$Gy'AcOmrLwwt
p$x;Seh-onQbfM?OjKbn'NwUAW -Np3fkz$FVwAUEa-wzWC -wQo-R!v -Mj?,SPiTyZ;o-opr$mOiPJEYD-
CfigkzD3p3?zvS;ADz;.y?o,ivCuC'zqHxcVT cHA
rT'Fd,SBMZyOslg!NXeF$sBe,juUzLq?w-wzP-h
ERjjxlgJzPbHxf$ q,q,KCDCU fqBOQT
SV&CW:xSVwZv'DG'NSPypDhKStKzC -$hslxIVzoivnp ,ethA:NCCGoi
tN!ljjP3fwJMwNelgUzzPGJlgihJ!d?q.d
pSPYgCuCJrIFtb
jQXg
pA.P LP,SPJi
DBcuBM:CixjJ$Jzkq,OLf3KLQLMGph$O 3DfiPHnXKuHMlyjxEiyZib3FaHV-oJa!zoc'XSP :CKGUhd?lgCOF$;;DTHZMlvvcmZA
D.?
```

7.4 The mathematical trick in self-attention

```
# toy example illustrating how matrix multiplication can be used for a "weighted aggregation"
torch.manual_seed(42)
a = torch.tril(torch.ones(3, 3))
a = a / torch.sum(a, 1, keepdim=True)
b = torch.randint(0,10,(3,2)).float()
c = a @ b
print('a=')
```

```
print(a)
print('--')
print('b=')
print(b)
print('--')
print('c=')
print(c)
```

```
a=
tensor([[1.0000, 0.0000, 0.0000],
        [0.5000, 0.5000, 0.0000],
        [0.3333, 0.3333, 0.3333]])
--
b=
tensor([[2., 7.],
        [6., 4.],
        [6., 5.]])
--
c=
tensor([[2.0000, 7.0000],
        [4.0000, 5.5000],
        [4.6667, 5.3333]])
```

```
# consider the following toy example:

torch.manual_seed(1337)
B,T,C = 4,8,2 # batch, time, channels
x = torch.randn(B,T,C)
x.shape
```

```
torch.Size([4, 8, 2])
```

7.5 Comprehension Questions — Weighted Aggregation via Matrix Multiplication

1. **Lower-triangular mask**: Why is the weight matrix `wei` lower-triangular? What would happen if you used a full matrix of ones instead?
2. **Three versions**: Versions 1, 2, and 3 compute the same result (`xbow`). What does each version gain over the previous one in terms of expressiveness or efficiency?
3. **Softmax version**: In version 3, `wei` is initialized to zeros, then `-inf` is placed in the upper triangle, then `softmax` is applied. Why does `softmax(-inf) = 0`? What is the role of this masking in the context of autoregressive generation?
4. **Uniform averaging**: All three versions compute a *uniform* average over past tokens. Why is this limiting, and what does the self-attention mechanism (version 4) add to fix it?

```
# We want x[b,t] = mean_{i<=t} x[b,i]
xbow = torch.zeros((B,T,C))
for b in range(B):
    for t in range(T):
```

```
        xprev = x[b,:t+1] # (t,C)
        xbow[b,t] = torch.mean(xprev, 0)
```

```
# version 2: using matrix multiply for a weighted aggregation
wei = torch.tril(torch.ones(T, T))
wei = wei / wei.sum(1, keepdim=True)
xbow2 = wei @ x # (B, T, T) @ (B, T, C) ----> (B, T, C)
torch.allclose(xbow, xbow2)
```

```
True
```

7.6 Comprehension Questions — Self-Attention

1. **Keys, Queries, Values**: In your own words, explain the role of each of the three projections (K, Q, V). What would happen if you used only one projection (i.e., set `key = query = value`)?
2. **Attention weights**: After computing `wei = q @ k.transpose(-2, -1)`, what is the shape of `wei` for a batch of size B, sequence length T, and head size C? What does entry `wei[b, t, s]` represent?
3. **Scaled dot-product**: Why do we divide `wei` by `sqrt(head_size)` before the softmax? What problem does this prevent? (Hint: look at cells 27–31.)
4. **Positional encoding**: The notes state that "attention has no notion of space." Why is positional encoding necessary, and where is it added in the full GPT model (cell 37)?
5. **Encoder vs Decoder**: What is the only code change needed to turn this decoder attention block into an encoder attention block? In what tasks would an encoder block be more appropriate?

```
# version 3: use Softmax
tril = torch.tril(torch.ones(T, T))
wei = torch.zeros((T,T))
wei = wei.masked_fill(tril == 0, float('-inf'))
wei = F.softmax(wei, dim=-1)
xbow3 = wei @ x
torch.allclose(xbow, xbow3)
```

```
True
```

```
# version 4: self-attention!
torch.manual_seed(1337)
B,T,C = 4,8,32 # batch, time, channels
x = torch.randn(B,T,C)

# let's see a single Head perform self-attention
head_size = 16
key = nn.Linear(C, head_size, bias=False)
query = nn.Linear(C, head_size, bias=False)
value = nn.Linear(C, head_size, bias=False)
k = key(x)   # (B, T, 16)
q = query(x) # (B, T, 16)
wei =  q @ k.transpose(-2, -1) # (B, T, 16) @ (B, 16, T) ---> (B, T, T)
```

```
tril = torch.tril(torch.ones(T, T))
#wei = torch.zeros((T,T))
wei = wei.masked_fill(tril == 0, float('-inf'))
wei = F.softmax(wei, dim=-1)

v = value(x)
out = wei @ v
#out = wei @ x

out.shape
```

```
torch.Size([4, 8, 16])
```

```
wei[0]
```

```
tensor([[1.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.1574, 0.8426, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.2088, 0.1646, 0.6266, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.5792, 0.1187, 0.1889, 0.1131, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.0294, 0.1052, 0.0469, 0.0276, 0.7909, 0.0000, 0.0000, 0.0000],
        [0.0176, 0.2689, 0.0215, 0.0089, 0.6812, 0.0019, 0.0000, 0.0000],
        [0.1691, 0.4066, 0.0438, 0.0416, 0.1048, 0.2012, 0.0329, 0.0000],
        [0.0210, 0.0843, 0.0555, 0.2297, 0.0573, 0.0709, 0.2423, 0.2391]],
       grad_fn=<SelectBackward0>)
```

Notes: - Attention is a **communication mechanism**. Can be seen as nodes in a directed graph looking at each other and aggregating information with a weighted sum from all nodes that point to them, with data-dependent weights. - There is no notion of space. Attention simply acts over a set of vectors. This is why we need to positionally encode tokens. - Each example across batch dimension is of course processed completely independently and never "talk" to each other - In an "encoder" attention block just delete the single line that does masking with `tril`, allowing all tokens to communicate. This block here is called a "decoder" attention block because it has triangular masking, and is usually used in autoregressive settings, like language modeling. - "self-attention" just means that the keys and values are produced from the same source as queries. In "cross-attention", the queries still get produced from x, but the keys and values come from some other, external source (e.g. an encoder module) - "Scaled" attention additional divides `wei` by 1/sqrt(head_size). This makes it so when input Q,K are unit variance, wei will be unit variance too and Softmax will stay diffuse and not saturate too much. Illustration below

```
k = torch.randn(B,T,head_size)
q = torch.randn(B,T,head_size)
wei = q @ k.transpose(-2, -1) * head_size**-0.5
```

```
k.var()
```

```
tensor(1.0449)
```

```
q.var()
```

```
tensor(1.0700)
```

```
wei.var()
```

```
tensor(1.0918)
```

7.7 Comprehension Questions — Layer Normalization

1. **BatchNorm vs LayerNorm**: The `LayerNorm1d` class normalizes over the *feature* dimension (axis 1) for each sample. BatchNorm normalizes over the *batch* dimension for each feature. Verify this with the outputs of cells 33 and 34: which axis has zero mean and unit std?

2. **Why LayerNorm in Transformers?** BatchNorm works well for CNNs but is problematic for sequence models. Identify at least two reasons why LayerNorm is preferred in Transformer architectures.

3. **Learnable parameters**: `LayerNorm1d` has `gamma` and `beta` parameters. What is their role? What would happen if you removed them (i.e., fixed `gamma=1, beta=0` without gradient)?

4. **Pre-norm vs Post-norm**: In the `Block` class (cell 37), LayerNorm is applied *before* the attention and feed-forward layers (`x = x + self.sa(self.ln1(x))`). This is called "pre-norm." The original "Attention is All You Need" paper used post-norm. What are the practical advantages of pre-norm for training deep networks?

```
torch.softmax(torch.tensor([0.1, -0.2, 0.3, -0.2, 0.5]), dim=-1)
```

```
tensor([0.1925, 0.1426, 0.2351, 0.1426, 0.2872])
```

```
torch.softmax(torch.tensor([0.1, -0.2, 0.3, -0.2, 0.5])*8, dim=-1) # gets too peaky, converges to one
```

```
tensor([0.0326, 0.0030, 0.1615, 0.0030, 0.8000])
```

7.8 Comprehension Questions — Full GPT Model

1. **Residual connections**: The `Block.forward` method uses `x = x + self.sa(self.ln1(x))` and `x = x + self.ffwd(self.ln2(x))`. What are residual (skip) connections, and why are they critical for training deep networks?

2. **Parameter count**: The model reports ~0.21M parameters with `n_embd=64, n_head=4, n_layer=4, vocab_size=65, block_size=32`. Break down where these parameters come from:

   - Token embedding table
   - Position embedding table
   - Each attention head (K, Q, V projections + output projection)
   - Each feed-forward block
   - Layer norms
   - Final LM head

3. **Feed-forward expansion**: The `FeedFoward` module expands from `n_embd` to `4 * n_embd` then back. Why use this expansion factor of 4? What role does the feed-forward layer play in a Transformer block?

4. **Overfitting**: At step 4999, train loss is 1.66 and val loss is 1.82. Is the model overfitting? What hyperparameters would you change to reduce the gap?

5. **Scaling up**: GPT-2 (117M params) achieves much better language modeling than this 0.2M model. Identify which hyperparameters (`n_embd`, `n_head`, `n_layer`, `block_size`) you would increase, and describe the memory/compute trade-offs of each.

6. **Extension — temperature sampling**: The `generate` method currently samples from the raw softmax distribution. How would you add a *temperature* parameter to control the randomness of generation? What does → 0 (greedy) vs → ∞ (uniform random) mean in practice?

```
class LayerNorm1d: # (used to be BatchNorm1d)

  def __init__(self, dim, eps=1e-5, momentum=0.1):
    self.eps = eps
    self.gamma = torch.ones(dim)
    self.beta = torch.zeros(dim)

  def __call__(self, x):
    # calculate the forward pass
    xmean = x.mean(1, keepdim=True) # batch mean
    xvar = x.var(1, keepdim=True) # batch variance
    xhat = (x - xmean) / torch.sqrt(xvar + self.eps) # normalize to unit variance
    self.out = self.gamma * xhat + self.beta
    return self.out

  def parameters(self):
    return [self.gamma, self.beta]

torch.manual_seed(1337)
module = LayerNorm1d(100)
x = torch.randn(32, 100) # batch size 32 of 100-dimensional vectors
x = module(x)
x.shape
```

```
torch.Size([32, 100])
```

```
x[:,0].mean(), x[:,0].std() # mean,std of one feature across all batch inputs
```

```
(tensor(0.1469), tensor(0.8803))
```

```
x[0,:].mean(), x[0,:].std() # mean,std of a single input from the batch, of its features
```

```
(tensor(-9.5367e-09), tensor(1.0000))
```

```
# French to English translation example:

# <--------- ENCODE ------------------><--------------- DECODE ----------------->
# les réseaux de neurones sont géniaux! <START> neural networks are awesome!<END>
```

7.8.1 Full finished code, for reference

You may want to refer directly to the git repo instead though.

```
import torch
import torch.nn as nn
from torch.nn import functional as F

# hyperparameters
batch_size = 16 # how many independent sequences will we process in parallel?
block_size = 32 # what is the maximum context length for predictions?
max_iters = 5000
eval_interval = 100
```

```
learning_rate = 1e-3
device = 'cuda' if torch.cuda.is_available() else 'cpu'
eval_iters = 200
n_embd = 64
n_head = 4
n_layer = 4
dropout = 0.0
# ------------

torch.manual_seed(1337)

# wget https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt
with open('input.txt', 'r', encoding='utf-8') as f:
    text = f.read()

# here are all the unique characters that occur in this text
chars = sorted(list(set(text)))
vocab_size = len(chars)
# create a mapping from characters to integers
stoi = { ch:i for i,ch in enumerate(chars) }
itos = { i:ch for i,ch in enumerate(chars) }
encode = lambda s: [stoi[c] for c in s] # encoder: take a string, output a list of integers
decode = lambda l: ''.join([itos[i] for i in l]) # decoder: take a list of integers, output a string

# Train and test splits
data = torch.tensor(encode(text), dtype=torch.long)
n = int(0.9*len(data)) # first 90% will be train, rest val
train_data = data[:n]
val_data = data[n:]

# data loading
def get_batch(split):
    # generate a small batch of data of inputs x and targets y
    data = train_data if split == 'train' else val_data
    ix = torch.randint(len(data) - block_size, (batch_size,))
    x = torch.stack([data[i:i+block_size] for i in ix])
    y = torch.stack([data[i+1:i+block_size+1] for i in ix])
    x, y = x.to(device), y.to(device)
    return x, y

@torch.no_grad()
def estimate_loss():
    out = {}
    model.eval()
    for split in ['train', 'val']:
        losses = torch.zeros(eval_iters)
        for k in range(eval_iters):
            X, Y = get_batch(split)
            logits, loss = model(X, Y)
            losses[k] = loss.item()
```

```
        out[split] = losses.mean()
    model.train()
    return out

class Head(nn.Module):
    """ one head of self-attention """

    def __init__(self, head_size):
        super().__init__()
        self.key = nn.Linear(n_embd, head_size, bias=False)
        self.query = nn.Linear(n_embd, head_size, bias=False)
        self.value = nn.Linear(n_embd, head_size, bias=False)
        self.register_buffer('tril', torch.tril(torch.ones(block_size, block_size)))

        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        B,T,C = x.shape
        k = self.key(x)   # (B,T,C)
        q = self.query(x) # (B,T,C)
        # compute attention scores ("affinities")
        wei = q @ k.transpose(-2,-1) * C**-0.5 # (B, T, C) @ (B, C, T) -> (B, T, T)
        wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf')) # (B, T, T)
        wei = F.softmax(wei, dim=-1) # (B, T, T)
        wei = self.dropout(wei)
        # perform the weighted aggregation of the values
        v = self.value(x) # (B,T,C)
        out = wei @ v # (B, T, T) @ (B, T, C) -> (B, T, C)
        return out

class MultiHeadAttention(nn.Module):
    """ multiple heads of self-attention in parallel """

    def __init__(self, num_heads, head_size):
        super().__init__()
        self.heads = nn.ModuleList([Head(head_size) for _ in range(num_heads)])
        self.proj = nn.Linear(n_embd, n_embd)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        out = torch.cat([h(x) for h in self.heads], dim=-1)
        out = self.dropout(self.proj(out))
        return out

class FeedFoward(nn.Module):
    """ a simple linear layer followed by a non-linearity """

    def __init__(self, n_embd):
        super().__init__()
        self.net = nn.Sequential(
```

```
            nn.Linear(n_embd, 4 * n_embd),
            nn.ReLU(),
            nn.Linear(4 * n_embd, n_embd),
            nn.Dropout(dropout),
        )

    def forward(self, x):
        return self.net(x)

class Block(nn.Module):
    """ Transformer block: communication followed by computation """

    def __init__(self, n_embd, n_head):
        # n_embd: embedding dimension, n_head: the number of heads we'd like
        super().__init__()
        head_size = n_embd // n_head
        self.sa = MultiHeadAttention(n_head, head_size)
        self.ffwd = FeedFoward(n_embd)
        self.ln1 = nn.LayerNorm(n_embd)
        self.ln2 = nn.LayerNorm(n_embd)

    def forward(self, x):
        x = x + self.sa(self.ln1(x))
        x = x + self.ffwd(self.ln2(x))
        return x

# super simple bigram model
class BigramLanguageModel(nn.Module):

    def __init__(self):
        super().__init__()
        # each token directly reads off the logits for the next token from a lookup table
        self.token_embedding_table = nn.Embedding(vocab_size, n_embd)
        self.position_embedding_table = nn.Embedding(block_size, n_embd)
        self.blocks = nn.Sequential(*[Block(n_embd, n_head=n_head) for _ in range(n_layer)])
        self.ln_f = nn.LayerNorm(n_embd) # final layer norm
        self.lm_head = nn.Linear(n_embd, vocab_size)

    def forward(self, idx, targets=None):
        B, T = idx.shape

        # idx and targets are both (B,T) tensor of integers
        tok_emb = self.token_embedding_table(idx) # (B,T,C)
        pos_emb = self.position_embedding_table(torch.arange(T, device=device)) # (T,C)
        x = tok_emb + pos_emb # (B,T,C)
        x = self.blocks(x) # (B,T,C)
        x = self.ln_f(x) # (B,T,C)
        logits = self.lm_head(x) # (B,T,vocab_size)

        if targets is None:
```

```
            loss = None
        else:
            B, T, C = logits.shape
            logits = logits.view(B*T, C)
            targets = targets.view(B*T)
            loss = F.cross_entropy(logits, targets)

        return logits, loss

    def generate(self, idx, max_new_tokens):
        # idx is (B, T) array of indices in the current context
        for _ in range(max_new_tokens):
            # crop idx to the last block_size tokens
            idx_cond = idx[:, -block_size:]
            # get the predictions
            logits, loss = self(idx_cond)
            # focus only on the last time step
            logits = logits[:, -1, :] # becomes (B, C)
            # apply softmax to get probabilities
            probs = F.softmax(logits, dim=-1) # (B, C)
            # sample from the distribution
            idx_next = torch.multinomial(probs, num_samples=1) # (B, 1)
            # append sampled index to the running sequence
            idx = torch.cat((idx, idx_next), dim=1) # (B, T+1)
        return idx

model = BigramLanguageModel()
m = model.to(device)
# print the number of parameters in the model
print(sum(p.numel() for p in m.parameters())/1e6, 'M parameters')

# create a PyTorch optimizer
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)

for iter in range(max_iters):

    # every once in a while evaluate the loss on train and val sets
    if iter % eval_interval == 0 or iter == max_iters - 1:
        losses = estimate_loss()
        print(f"step {iter}: train loss {losses['train']:.4f}, val loss {losses['val']:.4f}")

    # sample a batch of data
    xb, yb = get_batch('train')

    # evaluate the loss
    logits, loss = model(xb, yb)
    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    optimizer.step()
```

```
# generate from the model
context = torch.zeros((1, 1), dtype=torch.long, device=device)
print(decode(m.generate(context, max_new_tokens=2000)[0].tolist()))
```

```
0.209729 M parameters
step 0: train loss 4.4116, val loss 4.4022
step 100: train loss 2.6568, val loss 2.6670
step 200: train loss 2.5090, val loss 2.5058
step 300: train loss 2.4198, val loss 2.4340
step 400: train loss 2.3503, val loss 2.3567
step 500: train loss 2.2970, val loss 2.3136
step 600: train loss 2.2410, val loss 2.2506
step 700: train loss 2.2062, val loss 2.2198
step 800: train loss 2.1638, val loss 2.1871
step 900: train loss 2.1232, val loss 2.1494
step 1000: train loss 2.1020, val loss 2.1293
step 1100: train loss 2.0704, val loss 2.1196
step 1200: train loss 2.0382, val loss 2.0798
step 1300: train loss 2.0249, val loss 2.0640
step 1400: train loss 1.9922, val loss 2.0354
step 1500: train loss 1.9707, val loss 2.0308
step 1600: train loss 1.9614, val loss 2.0474
step 1700: train loss 1.9393, val loss 2.0130
step 1800: train loss 1.9070, val loss 1.9943
step 1900: train loss 1.9057, val loss 1.9871
step 2000: train loss 1.8834, val loss 1.9954
step 2100: train loss 1.8719, val loss 1.9758
step 2200: train loss 1.8582, val loss 1.9623
step 2300: train loss 1.8546, val loss 1.9517
step 2400: train loss 1.8410, val loss 1.9476
step 2500: train loss 1.8167, val loss 1.9455
step 2600: train loss 1.8263, val loss 1.9401
step 2700: train loss 1.8108, val loss 1.9340
step 2800: train loss 1.8040, val loss 1.9247
step 2900: train loss 1.8044, val loss 1.9304
step 3000: train loss 1.7963, val loss 1.9242
step 3100: train loss 1.7687, val loss 1.9147
step 3200: train loss 1.7547, val loss 1.9102
step 3300: train loss 1.7557, val loss 1.9037
step 3400: train loss 1.7547, val loss 1.8946
step 3500: train loss 1.7385, val loss 1.8968
step 3600: train loss 1.7260, val loss 1.8914
step 3700: train loss 1.7257, val loss 1.8808
step 3800: train loss 1.7204, val loss 1.8919
step 3900: train loss 1.7215, val loss 1.8788
step 4000: train loss 1.7146, val loss 1.8639
step 4100: train loss 1.7095, val loss 1.8724
step 4200: train loss 1.7079, val loss 1.8707
step 4300: train loss 1.7035, val loss 1.8502
step 4400: train loss 1.7043, val loss 1.8693
```

```
step 4500: train loss 1.6914, val loss 1.8522
step 4600: train loss 1.6853, val loss 1.8357
step 4700: train loss 1.6862, val loss 1.8483
step 4800: train loss 1.6671, val loss 1.8434
step 4900: train loss 1.6736, val loss 1.8415
step 4999: train loss 1.6635, val loss 1.8226

FlY BOLINGLO:
Them thrumply towiter arts the
muscue rike begatt the sea it
What satell in rowers that some than othis Marrity.

LUCENTVO:
But userman these that, where can is not diesty rege;
What and see to not. But's eyes. What?

JOHN MARGARET:
Than up I wark, what out, I ever of and love,
one these do sponce, vois I me;
But my pray sape to ries all to the not erralied in may.

BENVOLIO:
To spits as stold's bewear I would and say mesby all
on sworn make he anough
As cousins the solle, whose be my conforeful may lie them yet
nobe allimely untraled to be thre I say be,
Notham a brotes theme an make come,
And that his reach to the duke ento
the grmeants bell! and now there king-liff-or grief?

GLOUCESTER:
All the bettle dreene, for To his like thou thron!

MENENIUS:
Then, if I knom her all.
My lord, but terruly friend
Rish of the ploceiness and wilt tends sure?
Is you knows a fasir wead
That with him my spaut,
I shall not tas where's not, becomity; my coulds sting,
then the wit be dong to tyget our hereefore,
Who strop me, mend here, if agains, bitten, thy lack.
The but these it were is tus. For the her skeep the fasting. joy tweet Bumner:-
How the enclady: It you and how,
I am in him, And ladderle:
Their hand whose wife, it my hithre,
Roman and where sposs gives'd you.

TROMIOLANUS:
But livants you great, I shom mistrot come, for to she to lot
for smy to men ventry mehus. Gazise;
```

```
Full't were some the cause, and stouch set,
Or promises, which a kingsasted to your gove them; and sterrer,
And that wae love him.

BRUTUS:
You shape with these sweet.

CORTENGONO:
Lo, where 'twon elmes, 'morth young agres;
Sir, azavoust to striel accurded we missery sets crave.

ANGOLUM:
For is Henry to have gleise the dreason
That I ant shorfold wefth their servy in enscy.

ISABELLA:
O, I better you eyse such formfetrews.

BUCKINGHARENT:
Qead my lightle this righanneds flase them
Wam which an take was our some pleasurs,
Lovisoname to me, then fult me?--have it?

HENRY BOLINGBROY:
That wha
```